



Remaining useful life prediction based on a subdomain adaptation and contextual information compensation network

Zhiqing Xu^{ID}, Yee Wei Law^{ID}, Md Mizanur Rahman^{ID}, Raufdeen Rameezdeen^{ID}, Christopher W.K. Chow^{ID*}

Sustainable Infrastructure and Resource Management (SIRM), UniSA STEM, University of South Australia, Mawson Lakes, SA 5095, Australia

ARTICLE INFO

Keywords:

Remaining useful life prediction
Subdomain adaptation
Subdomain division
Contrastive learning

ABSTRACT

In recent years, *subdomain adaptation* (SDA) has been widely used to predict the *remaining useful life* (RUL) of bearings under different working conditions. Although SDA-based methods have achieved accurate predictions, most are still facing challenges, such as fixed subdomain boundaries, the assumption of symmetric domain structures, and ignoring a time-weighted mechanism. These limitations hinder further improvements in prediction accuracy. To address these issues, a *subdomain adaptation and contextual information compensation* (SDA-CIC) network was proposed that dynamically and adaptively divided subdomains in each training epoch to achieve precise SDA. To validate its effectiveness, hyperparameter analysis, ablation experiments, comparison experiments, and generalization experiments were performed. Experimental results show that the proposed SDA-CIC outperforms the state-of-the-art models.

1. Introduction

As essential components of industrial machinery, rolling bearings are subject to significant wear and tear, leading to inevitable failures of industrial equipment [1]. A survey by Thorsen et al. [2] revealed that bearing failures account for 51% of induction motor faults in the petrochemical industry. Such failures not only cause substantial financial losses but also compromise the reliability and stability of machinery. Consequently, accurately predicting the RUL of bearings is crucial for preventing failures and minimizing operational risks. RUL prediction methods are generally classified into two categories [3]: model-based methods and data-driven methods. Model-based methods rely on parametric modeling of the bearing degradation process, requiring extensive expert knowledge and experimental validation [4]. In contrast, data-driven methods leverage easily accessible sensor data rather than expert knowledge, directly extracting relevant degradation features and employing machine learning techniques to develop RUL prediction models [5].

Among data-driven methods, deep learning has recently gained significant attention for its ability to automatically handle large-scale and nonlinear data [6]. *Deep neural networks* (DNNs), such as *convolutional neural network* (CNN) [7], *long short-term memory* (LSTM) [8], and *Transformer* [9], have shown remarkable success in capturing bearing degradation information due to their excellent feature extraction capabilities. However, source domain data used for model training

and target domain data used for model testing tend to have different feature distributions [10], making it difficult to achieve accurate RUL prediction using only these advanced DNNs. To address the domain shift problem, researchers have introduced *domain adaptation* (DA), which can minimize the feature distribution discrepancies between the source and target domain data [11].

Since the introduction of DA, the combination of DNNs with a DA module has become a commonly adopted framework for RUL prediction models [12,13]. In this framework, DNNs aim to map the relationships between input data and their labels, while the DA module focuses on minimizing feature distribution discrepancies between the source and target domains. However, most DA-based prediction models only achieve global-scale domain alignment, as illustrated in Fig. 1(a), while ignoring the fine-grained feature distribution discrepancies within subdomains [14]. In real industrial settings, source and target domain data usually consist of multiple clusters or subdomains, each with different feature distributions [15]. Global-scale DA may not effectively handle these internal differences. To overcome this challenge, researchers have proposed SDA.

SDA involves dividing the source and target domains into different subdomains and minimizing the feature distribution discrepancies within each subdomain [16], as shown in Fig. 1(b). Although precise subdomain alignment improves the accuracy of RUL prediction across different domains, it still faces three limitations that hinder further improvements in prediction accuracy.

* Corresponding author.

E-mail address: Christopher.Chow@unisa.edu.au (C.W.K. Chow).

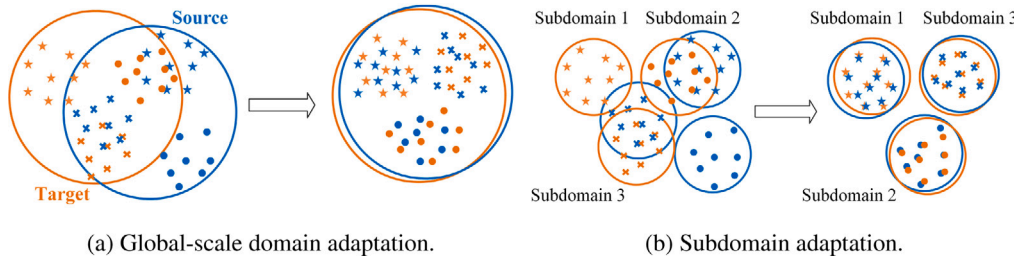


Fig. 1. Minimize the feature distribution discrepancies. (a) Global-scale domain adaptation may ignore fine-grained discrepancies within subdomains. (b) Subdomain adaptation effectively aligns each subdomain.

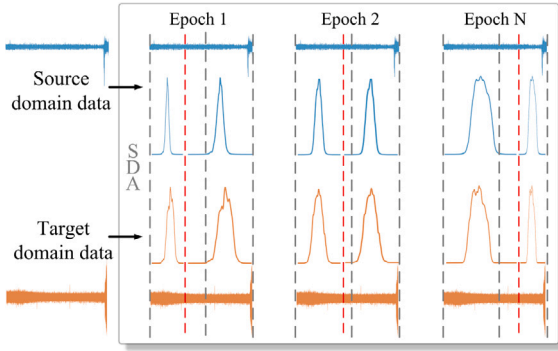


Fig. 2. Changes in subdomain boundaries during SDA. Blue and orange curves represent the probability density of source and target domain data, respectively. Black dashed lines indicate the fixed subdomain boundaries, while red dashed lines show the actual subdomain boundaries for the current epoch.

(1) **Fixed subdomain boundaries:** most methods pre-divide the source and target domain data into several subdomains before implementing SDA, and keep these subdomain boundaries fixed throughout the SDA process, as illustrated in Fig. 2. However, as feature distributions evolve dynamically during training, such fixed boundaries inevitably lead to imprecise subdomain alignment.

(2) **Symmetry assumption:** most methods assume a symmetric subdomain structure, where the source and target domains have the same number of subdomains [17]. As shown in Fig. 2, both domains contain two subdomains. However, due to varying working conditions, the source may have two subdomains while the target has three. When the symmetry assumption does not hold, the mismatch in the number of subdomains leads to alignment difficulties.

(3) **Lack of time-weighted mechanism:** most methods ignore the importance of time-weighted factors in the SDA process. Specifically, when predicting the RUL of bearings, not all time steps contribute equally. Time steps closer to failure typically contain more information related to bearing degradation [18]. Therefore, subdomains corresponding to these crucial time steps should be assigned higher weights during alignment.

To address these three limitations, this paper proposes a novel model called the SDA-CIC network, which includes three main functional components: ① a Downsampler, ② a subdomain matching module, and ③ a contextual information compensation (CIC) network. The Downsampler randomly downsamples a segment from the source domain in each training epoch. Next, the subdomain matching module uses the *dynamic time warping* (DTW) algorithm to identify the corresponding segment in the target domain that belongs to the same subdomain as the source segment, and then minimizes the feature distribution discrepancies between two matched segments. Finally, the CIC network compensates for the contextual information loss in the downsampled source segment caused by the Downsampler.

This paper focuses on the following improvements:

1. **Dynamic subdomain boundaries:** we introduce the SDA-CIC network, which dynamically adjusts subdomain boundaries during model training to adapt to evolving feature distributions, ensuring precise subdomain alignment.
2. **Adaptive subdomain matching:** SDA-CIC eliminates the constraint of symmetric subdomains by adaptively identifying the most relevant target domain segment for each sampled source domain segment, allowing flexibility in the domain structure.
3. **Focused degradation feature alignment:** SDA-CIC assigns higher weights to subdomains associated with the degradation stage, allowing the model to focus more attention on aligning key degradation-related features during SDA.

The rest of the paper is organized as follows. Section 2 introduces related works about DA and SDA in RUL prediction. Section 3 outlines the implementation details of the SDA-CIC network. Section 4 describes a case study using the proposed model to predict RUL, while evaluating the method in standard metrics. Section 5 discusses the choices of model components. Finally, Section 6 concludes.

2. Related works

In the context of bearing RUL prediction, there is already a large body of research literature on domain alignment. This section first provides a brief review of global-scale DA and local-scale SDA, and then compares the proposed SDA-CIC with related models to highlight the contribution of this research.

2.1. Domain adaptation for RUL prediction

In industrial settings, bearings inevitably exhibit different degradation trends due to varying working conditions [19]. Such variations lead to discrepancies in feature distributions between the source and target domains, making it difficult for DNNs to achieve accurate RUL prediction [20]. To address this challenge, global-scale DA techniques have been developed, including: ① metric-based methods, and ② adversarial-based methods [21].

Metric-based methods minimize discrepancies in feature distributions between the source and target domains within a high-dimensional feature space [22]. Common metrics for measuring feature distribution discrepancies include *maximum mean discrepancy* (MMD) [13], *multi-kernel maximum mean discrepancy* (MK-MMD) [23] and *Wasserstein distance* (WD) [24]. Among them, Cheng et al. [12] proposed an end-to-end baseline model, *transferable convolutional neural network* (TCNN), which extracts features through CNN and aligns the feature distributions using MK-MMD. Beyond the traditional source-available DA methods, Li et al. [25] proposed a novel *anti-forgetting source-free-based fault diagnosis* (AFSF-FD) model, which achieves source-free DA by iteratively optimizing pseudolabels and class prototypes under the nearest-neighbor constraint.

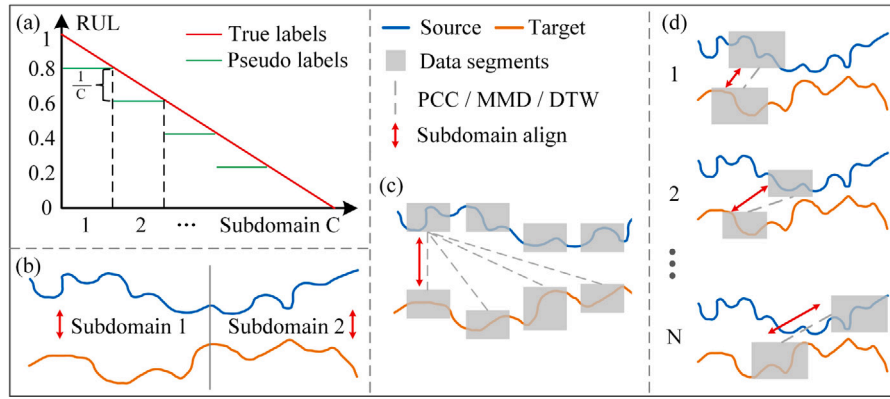


Fig. 3. Different strategies for subdomain division: (a) Discretize continuous labels into C discrete labels; (b) Predefine source domain into 2 subdomains; (c) Divide subdomains based on distance or similarity metrics; (d) Divide subdomains adaptively based on the proposed DTW.

Adversarial-based methods matches the feature distributions by modifying the feature representation itself rather than by performing a geometric transformation [26]. *Domain adversarial neural network* (DANN) proposed by Costa et al. [27], successfully learned domain invariant features through adversarial strategies. Building upon DANN, *multi-constrained domain adaptation* (MCDA) [28], and *mix-up domain adaptation* (MDAN) [29] have been developed specifically for RUL prediction tasks. Additionally, Zhuang et al. [30] developed an end-to-end model, *metric adversarial domain adaptation* (MADA), based on contrastive learning, which improves prediction accuracy by introducing a regularization term in the loss function. Despite the excellent predictive performance of these adversarial-based models, they often involve complex model architectures and require challenging hyperparameters tuning.

2.2. Subdomain adaptation for RUL prediction

SDA was originally developed for fault diagnosis (classification) tasks. A common strategy is to assign pseudolabels to target domain data and then minimize the discrepancies between source labels and predicted pseudolabels [31]. To improve diagnostic performance, Li et al. [32] proposed a *dynamic subdomain pseudolabel correction and adaptation* (DSPC-A) framework to alleviate the impact of misclassified pseudolabels.

Although pseudolabel-based SDA has achieved success in diagnosis, it cannot be directly applied to RUL prediction (regression) tasks, mainly due to the difference in label structure. In fault diagnosis, datasets usually contain explicit fault-type labels (e.g., inner, outer), which naturally define subdomains. In contrast, as shown in Fig. 3(a), RUL prediction involves continuous labels without fault types, making it difficult to clearly define the number of subdomains and divide subdomain boundaries. Therefore, a major challenge of applying SDA to RUL prediction lies in the subdomain division. According to subdomain division strategies, common SDA methods include: ① discretized continuous label-based methods, ② classification network-based methods, and ③ metric-based methods.

Discretized continuous label-based methods involve evenly dividing the continuous and normalized source domain labels into C intervals [33]. Each interval is processed by the floor function to assign a discrete subdomain label, as illustrated in Fig. 3(a). For example, Ding et al. [34] adopted this strategy in their *deep subdomain adaptive regression network* (DSARN), which divides source labels into 10 intervals, and then matches pseudolabels output from the target domain with discrete source labels. After clearly assigning the subdomain labels, researchers began to explore precise alignment strategies. Zhang et al. [16] developed a *variational LSTM local weighted subdomain adaptation network* (VLSTM-LWSAN) to achieve SDA via feature-weighted alignment. Lyu

et al. [35] further improved alignment by encouraging a clear separation between shared and private features, thereby aligning domain-specific shared features. Although these methods simplify subdomain division, they are inherently dependent on the assumption of discretization.

Classification network-based methods achieve SDA through a two-stage strategy. In the first stage, statistical or clustering algorithms are used to divide source domain data into several subdomains. For example, Yan et al. [36] used the Pettitt test to divide bearing degradation into five subdomains. Lu et al. [37] constructed Gaussian mixture models with several cluster numbers, and selected the one corresponding to the lowest Akaike information criterion value as the number of subdomains. In the second stage, classification networks are trained based on each subdomain data. These trained networks are then used to identify subdomains in the target domain and perform subdomain alignment. For example, Wu et al. [14] proposed a *deep subdomain adaptation network considering weighted multi-source domain* (DSAN-WM) that divides source domain data into two subdomains, trains classification networks for each, and uses these networks for subdomain division. Similarly, Dong et al. [13] introduced a *transferable LSTM multi-channel attention network* (TLMAN), which employs the same method to divide and align two subdomains, as shown in Fig. 3(b). Although these methods eliminate the need for the assumption of discretization, they rely heavily on expert knowledge for subdomain division.

Metric-based methods first divide the source and target domain data into several segments. Then, the similarity or distance between each source domain segment and all target domain segments is computed sequentially. The target domain segment with the highest similarity or the smallest distance to the current source domain segment is considered to belong to the same subdomain, as shown in Fig. 3(c). For example, Zhang et al. [38] proposed a *hierarchical adaptive multistage degradation network* (HAMDN), which first divides raw data into three segments and then uses MMD to measure the discrepancy between source and target domain segments. Zhuang et al. [39] proposed a *Graph-embedded subdomain adaptation network* (GSAN), which used local manifold distributions to divide subdomains and then employed a comprehensive similarity index, combining *Pearson correlation coefficient* (PCC) and Minkowski distance, to identify the most similar subdomain pairs within the source and target domains. Although these methods adaptively match subdomains, they still need to segment raw data before SDA, rather than dynamically dividing subdomains during model training.

2.3. Compared with related methods

To highlight the advantages of the proposed SDA-CIC, we compare it with four types of related models. These models exhibit limitations

Table 1
List of frequently used symbols and their definitions.

Symbol	Definition
x, y	Full-life data of any bearing and its corresponding labels;
x^S, x_{sub}^S	Source domain data and source subdomain data;
L^S, L^T	Lengths of the source and target domain data;
b^S, b^T	Starting position indices of the subdomains in the source and target domains;
$L_{\text{sub}}^S, L_{\text{sub}}^T$	Lengths of the source and target subdomain data;
$N_{\text{sub}}^S, N_{\text{sub}}^T$	Number of samples in the source and target subdomains;
w^S, w^T	Time-weighted factors for source and target subdomains;
$r^S, r_{\text{sub}}^S, r^T$	Feature representations for source domain, source subdomain, and target domain;
$\hat{y}_{\text{sub}}^S, \hat{y}_{\text{sub}}^T$	RUL prediction values for source and target subdomain data;
(r_{sub}^S, p^S)	Positive pairs;
$(r_{\text{sub}}^S, n_1^S)$	Negative pairs.

in capturing subdomain discrepancies, adapting to uneven data distributions, mitigating error accumulation, or handling mismatch errors. In the following, these limitations are analyzed in detail.

Compared to DA models, the proposed SDA-CIC achieves fine-grained SDA. Bearing degradation typically occurs in multiple stages, such as healthy stage, moderate degradation and severe degradation, each with distinct feature distributions. Traditional DA models use global alignment, which fails to accurately align the differences within each degradation stage.

Compared to discretized continuous label-based SDA models, the proposed SDA-CIC adaptively matches subdomains, meaning that only mismatching errors are likely to occur. In contrast, discretization divides the source data into uniform intervals, while feature distributions of real data are often uneven. Forced discretization can misassign similar data points to different subdomains or group dissimilar points together, introducing division errors. Moreover, the use of softmax and the floor function to generate pseudolabels may result in additional misclassified pseudolabels errors, further affecting the model's performance.

Compared to classification network-based SDA models, the proposed SDA-CIC is an end-to-end model. This allows mismatched errors from the subdomain matching module have the opportunity to be corrected by other networks (such as linear layers) under the overall optimization objective, reducing error propagation. In contrast, classification-based models rely on manual subdomain division in the first stage, where errors are directly passed to target domain predictions and cannot be optimized through joint training, leading to further error accumulation.

Compared to similarity metric-based SDA models, the proposed SDA-CIC uses random sampling to reduce matching errors caused by fixed segmentation. This strategy allows the model to cover data segments of varying lengths, ensuring that all segments have the opportunity to be sampled and aligned as subdomains during the training phase. Additionally, as an end-to-end model, SDA-CIC can gradually correct subdomain matching errors during the overall optimization process, minimizing error propagation and improving alignment accuracy.

3. Proposed method

As shown in Fig. 4, the proposed SDA-CIC consists of three main networks: ① the backbone RUL prediction network including an Encoder and a Predictor; ② the SDA network, which includes a Downsampler, a subdomain matching module, and a time-weighted MMD module; and ③ the CIC network. This section provides a detailed introduction to the function and implementation of each network. For clarity, all commonly used symbols in this section are listed in Table 1.

3.1. Problem description

The essence of RUL prediction is to learn a mapping $F: \mathcal{X} \rightarrow \mathcal{Y}$ that maps input features $x \in \mathcal{X}$ to corresponding RUL labels $y \in \mathcal{Y}$. Here, $\mathcal{X}$ and $\mathcal{Y}$ denote the feature space and label space, respectively.

Each input x represents a set of time-series data, while y is a scalar value between 0 and 1 (e.g., 0.3 represents 30% of RUL). However, the mapping F learned from one working condition often fails to be generalized to others due to discrepancies in feature distributions.

To overcome this challenge, DA is introduced to learn domain-invariant features in a high-dimensional Hilbert space $\mathcal{H}$, where the feature discrepancies between a labeled source domain dataset $\mathcal{D}_S = \{(x_i^S, y_i^S)\}_{i=1}^{N^S}$ and an unlabeled target domain dataset $\mathcal{D}_T = \{x_j^T\}_{j=1}^{N^T}$ are minimized. By sharing domain-invariant features and learning a mapping $F(x^S) \approx y^S$, the model can accurately infer the target domain prediction value $\hat{y}^T$ by mapping x^T into the shared feature space.

However, global-scale DA often overlooks fine-grained feature discrepancies within each domain. To address this issue, this research focuses on subdomain alignment. As shown in Fig. 4, input features x^S are first processed by the Downsampler to generate a segment x_{sub}^S . Then, an encoding function $\psi(\cdot)$ maps x_{sub}^S and x^T to feature representations $r_{\text{sub}}^S = \psi(x_{\text{sub}}^S)$ and $r^T = \psi(x^T)$. Next, a segment r_{sub}^T that is most similar to r_{sub}^S is identified from r^T to pair the subdomains. Finally, the matched subdomains r_{sub}^S and r_{sub}^T are processed by the Predictor to simultaneously learn a mapping $F(r_{\text{sub}}^S) \approx y_{\text{sub}}^S$ and align in $\mathcal{H}$ for predicting $\hat{y}^T$.

3.2. The backbone RUL prediction network

In SDA-based RUL prediction tasks, a key challenge is to learn domain-invariant features within subdomains. To address this challenge, a powerful and generalizable backbone network is required, which is expected to fulfill two basic requirements in terms of functionality: (1) the ability to learn the encoding function $\psi(\cdot)$, which encodes high-dimensional time-series data into low-dimensional feature representations that capture degradation patterns over time; and (2) the ability to learn the mapping F between the encoded feature representations and their corresponding labels, which is crucial for subsequent SDA and RUL prediction tasks.

To meet these two requirements, the proposed backbone network consists of two main components: ① an Encoder and ② a Predictor. The Encoder is responsible for feature encoding, while the Predictor maps the encoded feature representations to the final RUL prediction results.

Encoder adopts dilated CNN, which offer a larger receptive field and lower computational cost compared to standard CNN, making it particularly suitable for processing long time-series data [40]. As illustrated in Fig. 5, the proposed Encoder is achieved through 2 fully connected layers and 10 stacked *residual dilated convolution* (RDC) blocks. Each block comprises two 1-D convolution layers for feature extraction, two GeLU activation functions, and a residual structure to avoid gradient vanishing or explosion. The dilation parameters for the dilated convolutions in the i th block is set to 2^{i-1} , but all 1-D convolution layers within the same block share the same dilation parameters and kernel size.

Predictor takes the low-dimensional feature representations learned from the Encoder as input, and processes them through three fully connected layers. Each layer captures the mapping between the learned representations and their corresponding labels. The final output of the Predictor provides the RUL prediction results.

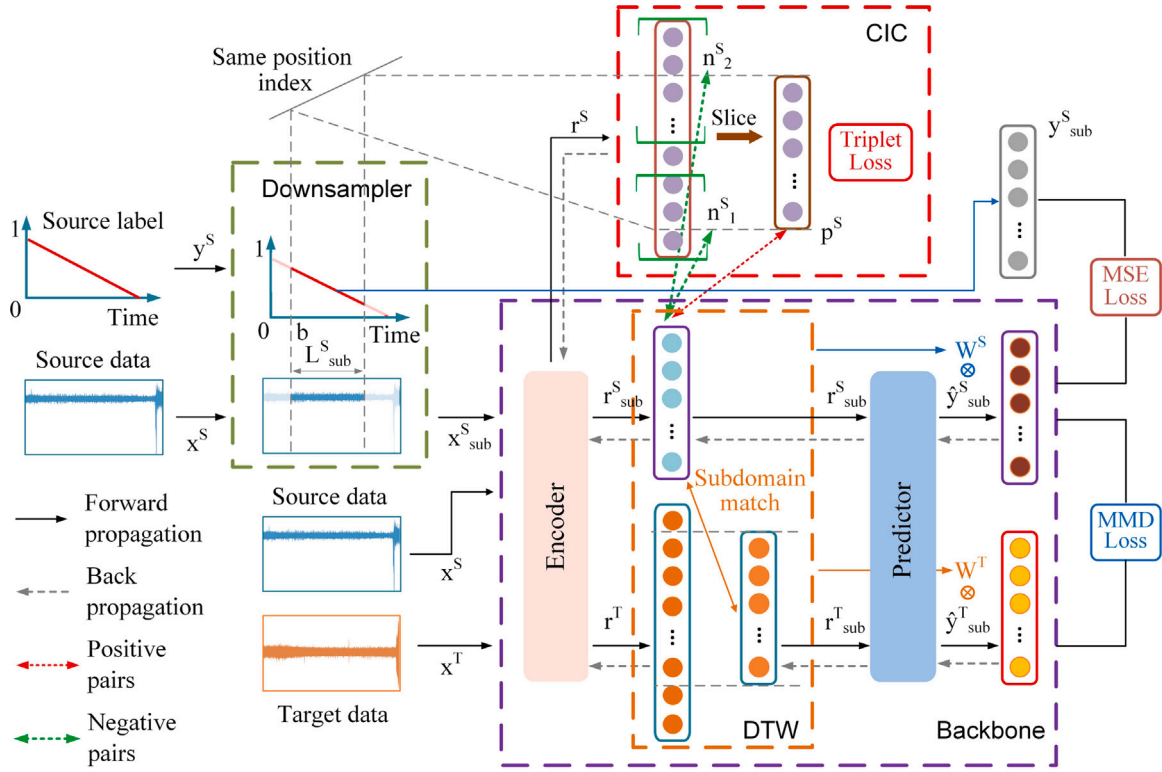


Fig. 4. The architecture of the proposed SDA-CIC model.

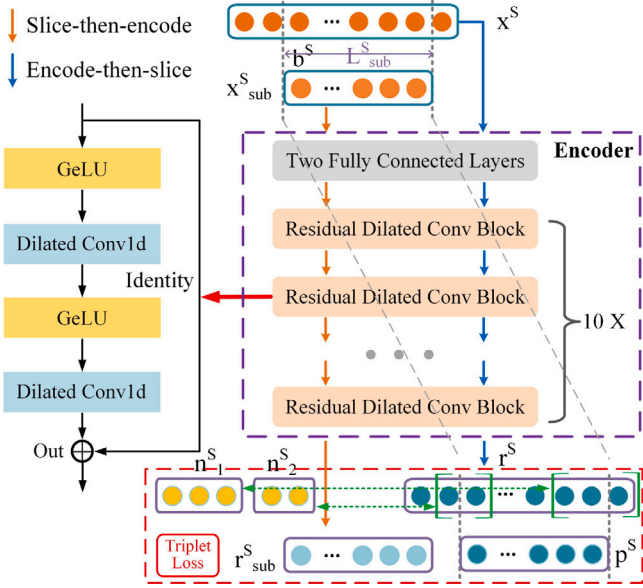


Fig. 5. The structure of Encoder and the workflow of Downsampler. The source domain data is encoded through two paths: the orange path indicates 'slice-then-encode', while the blue path indicates 'encode-then-slice'.

3.3. The subdomain adaptation network

To achieve fine-grained subdomain alignment, the SDA network is expected to fulfill three basic requirements in terms of functionality: (1) subdomain boundaries cannot be fixed, because feature distributions evolve dynamically during training; (2) the same subdomains in source and target domains should be adaptively matched rather than relying on symmetric assumptions or manual definitions; and (3)

distinct weights should be assigned to subdomains located in different degradation stages during alignment.

To meet the above three requirements, the proposed SDA network consists of three components: ① a Downsampler that dynamically provides subdomain boundaries at each training epoch, ② a subdomain matching module that adaptively identifies the corresponding subdomains, and ③ a time-weighted MMD module that emphasizes the alignment of key degradation-related features. The following parts detail the structure and function of each component.

The **Downsampler** randomly downsamples source domain data to generate segments with varying lengths in each epoch, which are then fed into the Encoder. Compared with feeding the entire time-series data into the Encoder at each epoch, dynamically slicing segments offers two advantages. First, it introduces diverse input features, serving as a form of data augmentation that mitigates overfitting to specific time steps and improves generalization to unseen data [41]. Second, this process inherently generates multiple subdomains with varying feature distributions, laying the foundation for dynamic subdomain alignment.

As illustrated in Fig. 4, in each training epoch, the Downsampler randomly slices a continuous segment from the source domain data x^S of length L^S , which is treated as the source subdomain data x_{sub}^S . This process is controlled by a starting index b^S and a randomly generated segment length L_{sub}^S ($b^S + L_{sub}^S < L^S$). The source subdomain data can be expressed as $x_{sub}^S = x^S[b^S : b^S + L_{sub}^S]$, and the corresponding label segment as $y_{sub}^S = y^S[b^S : b^S + L_{sub}^S]$, where $[b^S : b^S + L_{sub}^S]$ represents a slicing operation.

The **subdomain matching module** adaptively identifies a continuous segment from the target domain that belongs to the same subdomain as the source segment. To achieve such subdomain matching, DTW is applicable [24], which aligns two segments by minimizing the cumulative distance between them.

As shown in Fig. 4, the sliced x_{sub}^S and entire target domain data x^T are input into the Encoder for feature encoding, generating source domain feature representation segments r_{sub}^S and the entire target domain feature representations r^T , respectively. DTW is then used to match

subdomains by identifying a segment r_{sub}^T from r^T whose cumulative distance with r_{sub}^S is minimal. The cumulative distance is calculated as follows:

$$\text{CD} = \min_Q \sum_{k=1}^K d(q_k), \quad (1)$$

where $Q = (q_1, q_2, \dots, q_K)$ represents a warping path, with each time step $q_k = (i, j)$ corresponding to an alignment between $\{r_{\text{sub},i}^S\}_{i=1}^{N_{\text{sub}}^S}$ and $\{r_j^T\}_{j=1}^{N^T}$. Here, N_{sub}^S and N^T represent the number of samples in the source subdomain and target domain, respectively. K is the length of the warping path (the length of the searched segment in the target domain), which is adaptive and calculated by the algorithm to determine the optimal path length. To improve the inference speed, this paper fixes $K = L_{\text{sub}}^T = (b^S + L_{\text{sub}}^S) / L^S \cdot L^T - b^S / L^S \cdot L^T$, where L^S and L^T denote the lengths of the source and target domain data, respectively. The function $d(q_k)$ denotes the Euclidean distance between the aligned time steps.

After identifying the matched segment with the smallest cumulative distance from r^T , the corresponding position index b^T and segment length L_{sub}^T are determined. Next, the target subdomain representation $r_{\text{sub}}^T = r^T[b^T : b^T + L_{\text{sub}}^T]$ is sliced. Finally, r_{sub}^S and r_{sub}^T are fed into the Predictor to output RUL prediction results $\hat{y}_{\text{sub}}^S$ and $\hat{y}_{\text{sub}}^T$, respectively.

The **time-weighted MMD module** maps the RUL prediction results $\hat{y}_{\text{sub}}^S$ and $\hat{y}_{\text{sub}}^T$ into $\mathcal{H}$ to learn domain-invariant features. Since features at different time steps are not equally important, time-weighted factors are introduced to adjust the contribution of each subdomain in the alignment process. The time-weighted factor w is derived from the position index b and the subdomain length L_{sub} . The weights for the source and target subdomains are calculated as follows:

$$w^S = \frac{b^S + L_{\text{sub}}^S}{2L^S}, \quad (2)$$

$$w^T = \frac{b^T + L_{\text{sub}}^T}{2L^T}, \quad (3)$$

where b^S and b^T represent the starting position indices of the source and target subdomain data in their respective domains; L_{sub}^S and L_{sub}^T represent the lengths of the source and target subdomains, respectively; L^S and L^T represent the lengths of the source and target domains, respectively. The weights calculated for subdomains located in the healthy stage are close to 0, while those calculated for subdomains located in the degradation stage are close to 1. Therefore, the introduction of the time-weighted mechanism allows the alignment process to focus more attention on specific time steps that are considered to be more relevant to the degradation patterns.

The time-weighted MMD is calculated using a Gaussian kernel function $\phi(\cdot)$, which measures the feature distribution discrepancies between the $\hat{y}_{\text{sub}}^S$ and $\hat{y}_{\text{sub}}^T$.

$$\text{MMD}(\hat{y}_{\text{sub}}^S, \hat{y}_{\text{sub}}^T) = \left\| w^S \cdot \frac{1}{N_{\text{sub}}^S} \sum_{i=1}^{N_{\text{sub}}^S} \phi(\hat{y}_{\text{sub},i}^S) - w^T \cdot \frac{1}{N_{\text{sub}}^T} \sum_{j=1}^{N_{\text{sub}}^T} \phi(\hat{y}_{\text{sub},j}^T) \right\|_{\mathcal{H}}^2, \quad (4)$$

where w^S and w^T denote time-weighted factors for the source and target subdomains; $\hat{y}_{\text{sub},i}^S$ and $\hat{y}_{\text{sub},j}^T$ denote the predicted RUL values in the source and target subdomains, respectively; N_{sub}^S and N_{sub}^T denote the number of samples in the source and target subdomains, respectively; $\mathcal{H}$ represents the reproducing kernel Hilbert space.

3.4. The contextual information compensation network

As shown in Fig. 5, the source domain data x^S is processed in two parallel paths. In the first path, x^S is downsampled to obtain a source subdomain data x_{sub}^S along with its position index b^S and length L_{sub}^S .

The x_{sub}^S is then fed into the Encoder to obtain its feature representation r_{sub}^S . In the second path, x^S is directly fed into the Encoder to obtain its entire feature representation r^S , from which p^S is obtained by slicing at the same position index, i.e., $p^S = r^S[b^S : b^S + L_{\text{sub}}^S]$.

Although r_{sub}^S and p^S share the same position index, they are inherently different due to the order of slicing and encoding. Specifically, r_{sub}^S is obtained from a short data segment x_{sub}^S , which contains limited contextual information. In contrast, p^S is sliced from r^S , which is encoded from x^S with entire contextual information. In this research, the discrepancy between r_{sub}^S and p^S is expected to be minimized, so that the Encoder can infer global-like feature representations even when only data segments are provided. Ideally, the representations obtained by ‘slice-then-encode’ should be as similar as possible to those obtained by ‘encode-then-slice’.

To minimize this discrepancy, a CIC network based on contrastive learning framework is proposed. As a self-supervised learning network, it helps the Encoder compensate for the contextual information loss during the ‘slice-then-encode’ process without requiring any labels. The compensation process involves two steps: ① constructing positive-negative pairs. ② measuring the similarity of positive-negative pairs.

Constructing positive-negative pairs is essential to guide the Encoder to generate feature representations that retain contextual information. The core idea is to align the local segment and its corresponding slice of the full-sequence representation through similarity learning. In this research, the ‘encode-then-slice’ representation p^S and the ‘slice-then-encode’ representation r_{sub}^S are considered as positive pairs, as they share the same position index and length.

However, using only positive pairs may lead to representation collapse, where the Encoder always produces similar outputs and fails to learn meaningful differences. To avoid this, two non-overlapping representation segments n_1^S and n_2^S are randomly sampled from the entire representation r^S , as shown in Fig. 5. The pairs $(r_{\text{sub}}^S, n_1^S)$ and $(r_{\text{sub}}^S, n_2^S)$ are regarded as negative pairs. It is worth noting that p^S and r_{sub}^S have the same length, while the lengths of n_1^S and n_2^S may differ from r_{sub}^S because of the random sampling process.

Measuring the similarity of positive-negative pairs aims to assess the relationship between the constructed positive-negative pairs. The objective is to guide the Encoder to produce similar representations for positive pairs and dissimilar representations for negative pairs. Specifically, one optimization goal is to maximize the similarity or minimize the distance between positive pairs, such as p^S and r_{sub}^S . Conversely, another optimization goal is to minimize the similarity or maximize the distance between negative pairs, such as $(r_{\text{sub}}^S, n_1^S)$ and $(r_{\text{sub}}^S, n_2^S)$.

3.5. Total objective loss

To achieve accurate SDA-based RUL prediction, the proposed SDA-CIC network needs to be jointly trained and optimized under three optimization objectives: ① a regression loss L_R is adopted as the core objective to minimize the prediction errors between the ground truth and predicted RUL. ② a domain discrepancy loss L_D is required to align the feature distributions between the source and target subdomains for cross-domain prediction. ③ an information compensation loss L_T is introduced to compensate for the contextual information loss caused by the ‘slice-then-encode’ process, as described in Section 3.4.

For the first optimization objective, the *mean square error* (MSE) is used to define the regression loss L_R , which measures the average squared difference between the prediction value and the ground truth [28]. Minimizing L_R guides the predicted value of SDA-CIC to be closer to the actual value, enabling the model to learn the mapping from features to RUL in a supervised manner. The MSE function can be written as:

$$L_R = \frac{1}{N_{\text{sub}}^S} \sum_{i=1}^{N_{\text{sub}}^S} (\hat{y}_{\text{sub},i}^S - y_{\text{sub},i}^S)^2, \quad (5)$$

where $\hat{y}_{sub,i}^S$ and $y_{sub,i}^S$ are the predicted RUL values and the ground truth RUL values both from the source subdomain. N_{sub}^S is the number of samples in the source subdomain.

For the second optimization objective, the time-weighted MMD is employed to define the domain discrepancy loss L_D , which measures the feature distribution discrepancies between the source and target subdomains. Minimizing L_D encourages SDA-CIC to align feature distributions, enabling the transfer of knowledge from the labeled source domain to the unlabeled target domain. Moreover, time-weighted factors are integrated to highlight the temporal importance, helping the model to pay more attention to subdomains located in degradation stages. The loss function is defined as:

$$L_D = \text{MMD} \left(w^S \left\{ \hat{y}_{sub,i}^S \right\}_{i=1}^{N_{sub}^S}, w^T \left\{ \hat{y}_{sub,j}^T \right\}_{j=1}^{N_{sub}^T} \right), \quad (6)$$

where $\hat{y}_{sub}^S$ and $\hat{y}_{sub}^T$ denote the RUL prediction results of source and target subdomains. w^S and w^T denote the time-weighted factors for source and target subdomains;

For the third optimization objective, the triplet loss is perfectly suitable to define the information compensation loss L_T . The triplet loss is widely used in feature learning and metric learning tasks [42,43], where it compares three samples: an anchor (reference sample), a positive sample (similar to the anchor), and a negative sample (dissimilar to the anchor). The objective is to minimize the distance between the anchor and the positive sample while maximizing the distance between the anchor and the negative sample, thus encouraging the model to learn discriminative representations that preserve semantic relationships.

In this work, r_{sub}^S from the ‘slice-then-encode’ path is regarded as the anchor. p^S from the ‘encode-then-slice’ path serves as the positive sample. n_1^S and n_2^S are regarded as negative samples. By minimizing the distance between r_{sub}^S and p^S , while maximizing the distances between r_{sub}^S and negative samples n_1^S and n_2^S , SDA-CIC is encouraged to recover the complete semantic structure from representation segments. The loss function is defined as:

$$L_T = -\log(\sigma(r_{sub}^S \cdot p^S)) - \alpha \cdot \sum_{m=1}^M \log(\sigma(-r_{sub}^S \cdot n_m^S)), \quad (7)$$

where σ refers to the sigmoid function. α is used to balance the contribution of the negative pairs, and it is set to 0.3 in this paper. M represents the number of negative pairs and it is set to 2 in this paper.

The total loss of the proposed SDA-CIC is defined as:

$$\begin{aligned} \mathcal{L} = & L_R + \mu \cdot L_D + \lambda \cdot L_T = \frac{1}{N_{sub}^S} \sum (\hat{y}_{sub}^S - y_{sub}^S)^2 \\ & + \mu \cdot \text{MMD}(w^S \cdot \hat{y}_{sub}^S, w^T \cdot \hat{y}_{sub}^T) \\ & - \lambda \cdot \left[\log(\sigma(r_{sub}^S \cdot p^S)) + \alpha \cdot \sum_{m=1}^M \log(\sigma(-r_{sub}^S \cdot n_m^S)) \right], \end{aligned} \quad (8)$$

where μ and λ denote the trade-off parameters, both set to 0.1.

Once the total loss function $\mathcal{L}$ is defined, the optimal parameters of the proposed SDA-CIC can be searched by optimizing the total loss function and the calculation steps are summarized in Algorithm 1.

4. Case study

To demonstrate that the proposed SDA-CIC network can achieve high RUL prediction accuracy, a case study first identifies the optimal hyperparameter combination of the proposed SDA-CIC through the grid search. Ablation and comparison experiments are then conducted to evaluate the RUL prediction accuracy using the PRONOSTIA dataset, with three evaluation metrics introduced for quantification. Finally, the generalization performance of the SDA-CIC is validated using the XJTU-SY dataset.

Algorithm 1 SDA-CIC.

```

procedure PRE-TRAIN(raw data)
  for  $x^S$  in raw data do
    // Downsampler :
     $b^S, L_{sub}^S \leftarrow$  random generation;
     $x_{sub}^S \leftarrow$  slice subdomain data  $x^S[b^S : b^S + L_{sub}^S]$ ;
    // Encoder :
     $r_{sub}^S, r^S \leftarrow$  extract features  $x_{sub}^S, x^S$ ;
    // Calculate loss :
     $L_T \leftarrow \text{TripletLoss}(r^S, r_{sub}^S, b^S, L_{sub}^S)$ ;
    // Update Encoder parameters using  $L_T$ 
  end for
end procedure

procedure MAIN(raw data)
  for  $x^S, x^T, y^S$  in raw data do
    // Downsampler :
     $b^S, L_{sub}^S \leftarrow$  random generation;
     $x_{sub}^S \leftarrow$  slice subdomain data  $x^S[b^S : b^S + L_{sub}^S]$ ;
     $y_{sub}^S \leftarrow$  slice subdomain label  $y^S[b^S : b^S + L_{sub}^S]$ ;
    // Trained Encoder :
     $r_{sub}^S, r^S, r^T \leftarrow$  extract features  $x_{sub}^S, x^S, x^T$ ;
    // DTW :
     $r_{sub}^S, r_{sub}^T \leftarrow$  match subdomains  $r_{sub}^S, r_{sub}^T$ ;
     $w^S, w^T \leftarrow$  calculate weights  $b^S, L_{sub}^S, L^S, L^T$ ;
    // Predictor :
     $\hat{y}_{sub}^S, \hat{y}_{sub}^T \leftarrow$  FC layers  $r_{sub}^S, r_{sub}^T$ ;
    // Calculate loss :
     $L_R \leftarrow \text{MSELoss}(\hat{y}_{sub}^S, y_{sub}^S)$ ;
     $L_D \leftarrow \text{MMDLoss}(\hat{y}_{sub}^S, \hat{y}_{sub}^T, w^S, w^T)$ ;
     $L_T \leftarrow \text{TripletLoss}(r^S, r_{sub}^S, b^S, L_{sub}^S)$ ;
     $\mathcal{L} \leftarrow L_R + \mu L_D + \lambda L_T$ ;
    // Update model parameters using  $\mathcal{L}$ 
  end for
end procedure

function TRIPLETLOSS( $r^S, r_{sub}^S, b^S, L_{sub}^S$ )
  // Construct positive-negative pairs:
   $p^S \leftarrow$  slice representations  $r^S[b^S : b^S + L_{sub}^S]$ ;
   $n_1^S, n_2^S \leftarrow$  random negative pairs  $r^S$ ;
  // Calculate Triplet loss :
   $L_T \leftarrow$  measure similarity  $p^S, r_{sub}^S, n_1^S, n_2^S$ ;
  return  $L_T$ 
end function

```

Table 2

The description of PRONOSTIA [44] and XJTU-SY [45] datasets.

Datasets	Conditions	Load (N)	Speed (rpm)	Bearings
PRONOSTIA	C1	4000	1800	Bearing 1_1 ~ 1_7
	C2	4200	1650	Bearing 2_1 ~ 2_7
	C3	5000	1500	Bearing 3_1 ~ 3_3
XJTU-SY	C1	12 000	2100	Bearing 1_1 ~ 1_5
	C2	11 000	2250	Bearing 2_1 ~ 2_5
	C3	10 000	2400	Bearing 3_1 ~ 3_5

4.1. Data description

This study adopts two public bearing datasets, PRONOSTIA and XJTU-SY, both collected from acceleration test beds and recording vibration signals at a sampling frequency of 25600 Hz. In the PRONOSTIA dataset, 17 bearings are tested under various rotational speeds and radial loads, transitioning from healthy states to faulty states within a few hours, as presented in Table 2. Since each recording lasts for 0.1 s,

Table 3
Two sets of transfer tasks based on the PRONOSTIA [44] and XJTU-SY [45] datasets.

Datasets	Tasks	Conditions	Training bearings	Test bearings
PRONOSTIA	A1	C1 $\rightarrow$ C2	Labeled: Bearing 1_1; Unlabeled: Bearing 2_1	Bearing 2_6
	A2	C1 $\rightarrow$ C3	Labeled: Bearing 1_1; Unlabeled: Bearing 3_1	Bearing 3_3
	A3	C2 $\rightarrow$ C1	Labeled: Bearing 2_1; Unlabeled: Bearing 1_1	Bearing 1_3
	A4	C2 $\rightarrow$ C3	Labeled: Bearing 2_1; Unlabeled: Bearing 3_1	Bearing 3_3
	A5	C3 $\rightarrow$ C1	Labeled: Bearing 3_1; Unlabeled: Bearing 1_1	Bearing 1_3
	A6	C3 $\rightarrow$ C2	Labeled: Bearing 3_1; Unlabeled: Bearing 2_1	Bearing 2_6
XJTU-SY	B1	C1 $\rightarrow$ C2	Labeled: Bearing 1_1; Unlabeled: Bearing 2_1	Bearing 2_3
	B2	C1 $\rightarrow$ C3	Labeled: Bearing 1_1; Unlabeled: Bearing 3_1	Bearing 3_3
	B3	C2 $\rightarrow$ C1	Labeled: Bearing 2_1; Unlabeled: Bearing 1_1	Bearing 1_3
	B4	C2 $\rightarrow$ C3	Labeled: Bearing 2_1; Unlabeled: Bearing 3_1	Bearing 3_3
	B5	C3 $\rightarrow$ C1	Labeled: Bearing 3_1; Unlabeled: Bearing 1_1	Bearing 1_3
	B6	C3 $\rightarrow$ C2	Labeled: Bearing 3_1; Unlabeled: Bearing 2_1	Bearing 2_3

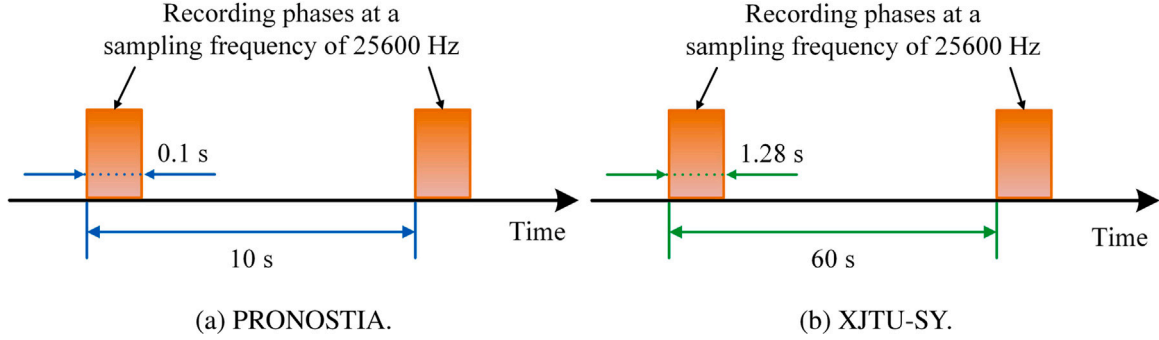


Fig. 6. Sampling strategy for the PRONOSTIA and XJTU-SY datasets.

the total number of data points per recording is $25600 \times 0.1 = 2560$. Data was recorded every 10 s, as shown in Fig. 6(a).

The XJTU-SY dataset consists of 15 run-to-failure bearings, as presented in Table 2. Since each recording lasts 1.28 s, the total number of data points per recording is $25600 \times 1.28 = 32768$. Data was recorded every 60 s, as shown in Fig. 6(b).

To comprehensively evaluate the performance of the proposed SDA-CIC, two sets of transfer tasks are designed, as shown in Table 3. The PRONOSTIA dataset is designated as Task A for hyperparameter tuning, ablation experiments, and comparison experiments, while the XJTU-SY dataset is designated as Task B for generalization experiments.

4.2. Model design and training

All experiments were conducted using Python 3.8 and PyTorch 1.13.0 on a CPU-based Windows 10 system. The key libraries used include NumPy 1.24.3 and scikit-learn 1.3.0. The proposed SDA-CIC is trained in two stages: a self-supervised pre-training stage to initialize the Encoder, followed by a main training stage to jointly optimize the Encoder and the Predictor for RUL prediction. In this part, the training strategies, loss functions, and hyperparameter configurations used in each stage are detailed.

Pre-training the Encoder: To enable the Encoder to generate informative representations before implementing downstream tasks, such as RUL prediction and SDA, it can be pretrained in an unsupervised manner based on the triplet loss using any unlabeled data, as shown in Fig. 5. The hyperparameter settings used for pre-training are summarized in Table 4. In this configuration, the batch size is set to 1, ensuring that the full-life data of one bearing is fed into the Encoder during each training epoch. The learning rate scheduler adopts the StepLR strategy, with a step size of 50 and a multiplication ratio of 0.1, meaning the learning rate is multiplied by 0.1 every 50 epochs. In the triplet loss, the number of negative pairs is set to 2, and the contribution weight of negative pairs is set to 0.3, both empirically determined to balance the representation discriminability and training stability.

Table 4
Hyperparameter settings for both pre-training and main training stages.

Hyperparameters	Settings
Optimizer	Adam
Initial learning rate	0.001
Scheduler	StepLR (50, 0.1)
Batch size	1
Max epochs	200
Number of negative pairs	2
Weight of negative pairs	0.3

Main training for SDA-CIC: After the Encoder has been pretrained, the main training stage for SDA-CIC begins. In this stage, the Predictor and pretrained Encoder are jointly optimized using the MSE loss, the MMD loss, and the triplet loss, as shown in Algorithm 1. The trade-off parameters μ and λ for the MMD loss and the triplet loss in Eq. (8), are set to 0.1 and 0.1. All other configurations remain the same as those used during the pre-training stage, as listed in Table 4.

To clearly illustrate the architecture of the SDA-CIC, including the input and output shapes of each layer, along with the corresponding parameters and operations, Bearing 1_3 from the PRONOSTIA dataset is used as an example. By tracing the data flow from input to output, a step-by-step illustration is provided to show how the data shapes change throughout the SDA-CIC, as shown in Table 5.

Specifically, Bearing 1_3 contains 2375 time steps over its full lifespan, with each time step containing 2560 features (i.e., 2560 data points per sample), as detailed in Fig. 6(a). Therefore, the input data can be represented as a matrix of size (2375×2560) . This input is first fed to the Downsampler, where a data segment is randomly selected in each training epoch. For example, if a segment of 1500 time steps is sampled in the current epoch, the resulting input shape becomes (1500×2560) . Next, this downsampled data segment is fed into the Encoder, which consists of 2 linear layers followed by 10 RDC blocks. The data shape changes through the 2 linear layers as follows: $(1500 \times 2560) \rightarrow (1500 \times 256)$. After reshaping to match the RDC

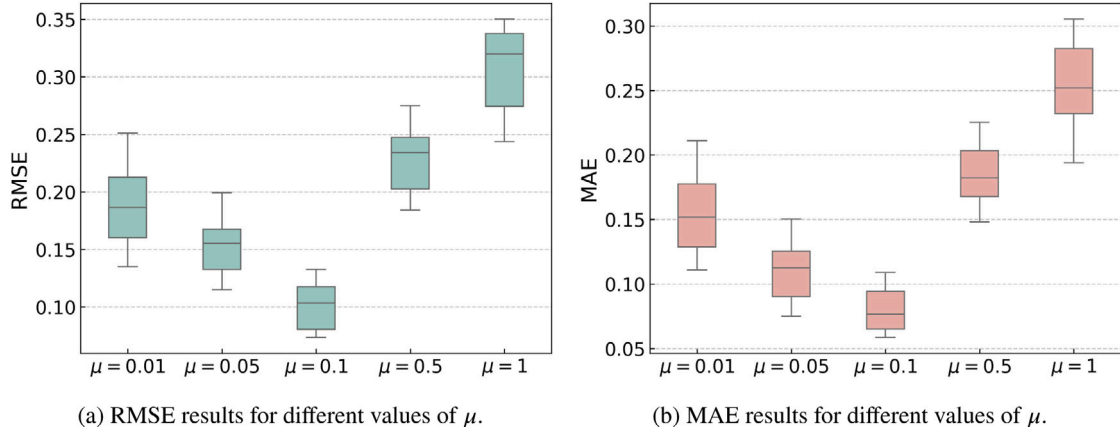
Fig. 7. RMSE and MAE results for different values of μ with $\lambda = 0.1$.

Table 5

The architecture parameters and operations of each layer as Bearing_1_3 passes through the SDA-CIC.

Module	Type	Shape	In/out features	Kernel	Dilation	Padding
Downsampler	Downsampling	(2375, 2560) → (1500, 2560)	–	–	–	–
Encoder	Linear 1	(1500, 2560) → (1500, 1024)	2560/1024	–	–	–
	Linear 2	(1500, 1024) → (1500, 256)	1024/256	–	–	–
	Unsqueeze	(1500, 256) → (1, 1500, 256)	–	–	–	–
	Transpose	(1, 1500, 256) → (1, 256, 1500)	–	–	–	–
	RDC Block 1	(1, 256, 1500) → (1, 256, 1500)	256/256	3	1	1
	RDC Block 2	(1, 256, 1500) → (1, 256, 1500)	256/256	3	2	2
	RDC Block 3	(1, 256, 1500) → (1, 256, 1500)	256/256	3	4	4
	RDC Block 4	(1, 256, 1500) → (1, 256, 1500)	256/256	3	8	8
	RDC Block 5	(1, 256, 1500) → (1, 256, 1500)	256/256	3	16	16
	RDC Block 6	(1, 256, 1500) → (1, 256, 1500)	256/256	3	32	32
	RDC Block 7	(1, 256, 1500) → (1, 256, 1500)	256/256	3	64	64
	RDC Block 8	(1, 256, 1500) → (1, 256, 1500)	256/256	3	128	128
	RDC Block 9	(1, 256, 1500) → (1, 256, 1500)	256/256	3	256	256
	RDC Block 10	(1, 256, 1500) → (1, 320, 1500)	256/320	3	512	512
Predictor	Transpose	(1, 320, 1500) → (1, 1500, 320)	–	–	–	–
	Squeeze	(1, 1500, 320) → (1500, 320)	–	–	–	–
	Linear 1	(1500, 320) → (1500, 128)	320/128	–	–	–
	Linear 2	(1500, 128) → (1500, 32)	128/32	–	–	–
	Linear 3	(1500, 32) → (1500, 1)	32/1	–	–	–

input format, the shape becomes $(1 \times 256 \times 1500)$ and is further transformed by the 10 RDC blocks to $(1 \times 320 \times 1500)$. Finally, the learned representations are fed to the Predictor, which consists of 3 linear layers for RUL prediction, where the data shape changes from (1500×320) to (1500×1) .

4.3. Evaluation metrics for RUL prediction

The accuracy of RUL prediction is measured using three types of prediction metrics, namely RMSE, MAE, and a Score function commonly adopted by researchers in the prognostic challenges [24]. These metrics are defined as follows:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\tilde{y}_i - y_i)^2}, \quad (9)$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\tilde{y}_i - y_i|, \quad (10)$$

$$\text{Score} = \frac{1}{N-1} \sum_{i=1}^{N-1} A_i, \quad (11)$$

where:

$$A_i = \begin{cases} e^{-\ln(0.5) \cdot (Er_i/5)} & \text{if } Er_i \leq 0, \\ e^{+\ln(0.5) \cdot (Er_i/20)} & \text{if } Er_i > 0, \end{cases} \quad (12)$$

$$Er_i = \frac{y_i - \tilde{y}_i}{y_i} \times 100, \quad (13)$$

where y_i and $\tilde{y}_i$ are the actual RUL and the predicted RUL respectively, N is the length of the testing data. When calculating Eq. (11), data with label 0 should be omitted. The higher the score function, the better the prediction performance.

4.4. Comparison of hyperparameters settings

This section discusses the impact of hyperparameters μ and λ from Eq. (8) in terms of RUL prediction accuracy. Given the sensitivity of the proposed SDA-CIC to μ and λ , a grid search method is conducted over the range $[0.01, 0.05, 0.1, 0.5, 1]$ to evaluate their effects. The preliminary results suggest that the optimal combination is $\mu = 0.1$ and $\lambda = 0.1$. To verify that this combination is not a local optimum, further experiments are performed by fixing one parameter while varying the other. Specifically, λ is fixed at 0.1 while μ ranges from $[0.01, 0.05, 0.1, 0.5, 1]$, and then μ is fixed at 0.1 while varying λ within the same range. To reduce the effect of randomness, all experiments are repeated 10 times, and the results are presented in Figs. 7 and 8.

The detailed results shown in Figs. 7 and 8, illustrate the trends for different values of μ and λ in terms of RMSE and MAE. When λ is fixed at 0.1, increasing μ from 0.01 to 0.1 improves the prediction accuracy. A small μ places insufficient emphasis on the L_D in Eq. (8), leaving a significant discrepancy between the source and target feature distributions, which harms the prediction. Increasing μ from 0.01 to 0.1 strengthens L_D , making the distributions of the source and target domains closer, thereby improving the prediction accuracy. However,

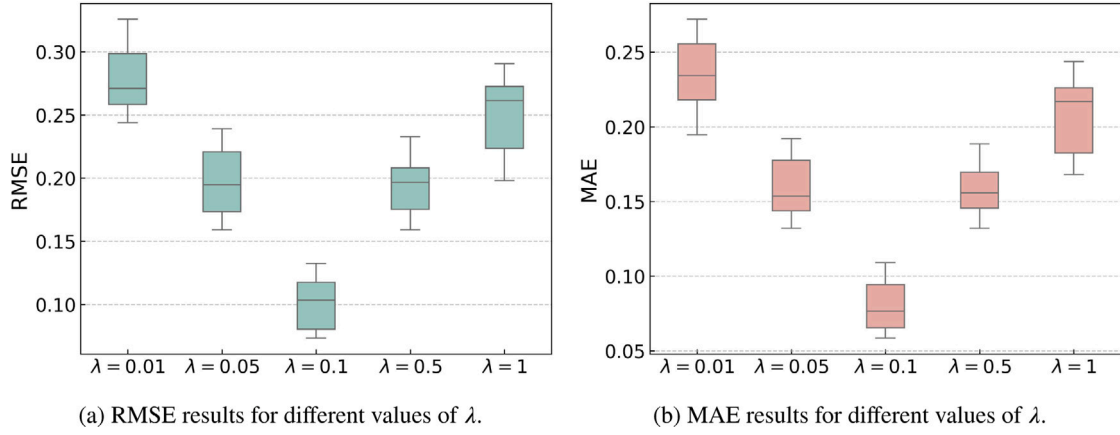
Fig. 8. RMSE and MAE results for different values of λ with $\mu = 0.1$.

Table 6
The RUL prediction metrics of Models A-D for the six transfer tasks in Table 3.

Tasks	Metrics	Model A	Model B	Model C	Model D
		w/o DS	w/o TW	w/o CIC	SDA-CIC
A1	MAE	0.1942 $\pm$ 0.0188	0.1001 $\pm$ 0.0379	0.1435 $\pm$ 0.0166	0.0615 $\pm$ 0.0102
	RMSE	0.2286 $\pm$ 0.0255	0.1217 $\pm$ 0.0459	0.1714 $\pm$ 0.0249	0.0734 $\pm$ 0.0116
	Score	0.2806 $\pm$ 0.0245	0.4285 $\pm$ 0.0873	0.3448 $\pm$ 0.0156	0.5207 $\pm$ 0.0229
A2	MAE	0.2148 $\pm$ 0.0753	0.1089 $\pm$ 0.0437	0.1443 $\pm$ 0.0208	0.0762 $\pm$ 0.0134
	RMSE	0.2571 $\pm$ 0.0915	0.1326 $\pm$ 0.0488	0.1798 $\pm$ 0.0239	0.1015 $\pm$ 0.0203
	Score	0.2698 $\pm$ 0.0806	0.4746 $\pm$ 0.1237	0.3748 $\pm$ 0.0225	0.5443 $\pm$ 0.0362
A3	MAE	0.1824 $\pm$ 0.0700	0.1116 $\pm$ 0.0398	0.1598 $\pm$ 0.0368	0.0585 $\pm$ 0.0107
	RMSE	0.2247 $\pm$ 0.0832	0.1426 $\pm$ 0.0446	0.2085 $\pm$ 0.0477	0.0735 $\pm$ 0.0110
	Score	0.3016 $\pm$ 0.1245	0.4360 $\pm$ 0.0838	0.3305 $\pm$ 0.0442	0.5536 $\pm$ 0.0404
A4	MAE	0.1909 $\pm$ 0.0197	0.1198 $\pm$ 0.0622	0.1320 $\pm$ 0.0218	0.0686 $\pm$ 0.0102
	RMSE	0.2319 $\pm$ 0.0268	0.1590 $\pm$ 0.0733	0.1457 $\pm$ 0.0305	0.0861 $\pm$ 0.0112
	Score	0.2942 $\pm$ 0.0216	0.4379 $\pm$ 0.1285	0.3913 $\pm$ 0.0288	0.5508 $\pm$ 0.0237
A5	MAE	0.2110 $\pm$ 0.0280	0.1209 $\pm$ 0.0245	0.1628 $\pm$ 0.0274	0.0769 $\pm$ 0.0169
	RMSE	0.2560 $\pm$ 0.0374	0.1609 $\pm$ 0.0323	0.2097 $\pm$ 0.0286	0.1052 $\pm$ 0.0233
	Score	0.2527 $\pm$ 0.0151	0.3692 $\pm$ 0.0411	0.3083 $\pm$ 0.0308	0.4883 $\pm$ 0.0474
A6	MAE	0.2272 $\pm$ 0.0177	0.1581 $\pm$ 0.0214	0.1696 $\pm$ 0.0277	0.0710 $\pm$ 0.0251
	RMSE	0.2613 $\pm$ 0.0251	0.1866 $\pm$ 0.0256	0.1986 $\pm$ 0.0404	0.0844 $\pm$ 0.0275
	Score	0.2322 $\pm$ 0.0070	0.3546 $\pm$ 0.0217	0.3147 $\pm$ 0.0323	0.5524 $\pm$ 0.0648

further increasing μ to 1 significantly reduces the accuracy. This is because the L_D dominates the optimization process, causing the model to focus too much on distribution alignment while neglecting the main RUL prediction task, resulting in a decrease in prediction accuracy.

Similarly, when μ is fixed at 0.1, increasing λ from 0.01 to 0.1 significantly improves prediction accuracy. This improvement stems from the fact that L_T in Eq. (8) compensates for the contextual information loss caused by the ‘slice-then-encode’ process. However, when λ exceeds 0.1, the accuracy decreases because L_T dominates the optimization process, making the model focus too much on information compensation and neglecting both RUL prediction and domain alignment.

4.5. Ablation experiments using SDA-CIC

The proposed SDA-CIC consists of three main functional components: ① the Down sampler (DS); ② the time-weighted (TW) MMD module; and ③ the CIC network. To assess the contribution of each component, ablation experiments are conducted across six transfer tasks based on the PRONOSTIA dataset, as described in Table 3. The configurations of models for the ablation experiments are as follows:

- w/o DS: Model A excludes the Down sampler. The complete source and target domain data are directly fed into the backbone network, meaning that Model A is a traditional global-scale DA model.
- w/o TW: Model B excludes the time-weighted mechanism from the TW-based MMD module, meaning that all subdomains are assigned equal weights during subdomain alignment.
- w/o CIC: Model C excludes the CIC network. Source domain segments with missing contextual information are fed into the backbone network for SDA and RUL prediction, without any information compensation.
- Model D: The proposed SDA-CIC network.

To minimize the effect of randomness, all experiments are repeated 10 times. The RUL prediction curves for six transfer tasks are presented in Fig. 9. Table 6 shows the prediction accuracy of these four models in terms of RMSE, MAE, and Score, while Fig. 10 illustrates the average RUL prediction accuracy of these four models.

As shown in Fig. 10, Model A exhibits the highest prediction errors (MAE and RMSE) and the lowest Score. Additionally, the prediction curves of Model A shown in Fig. 9 deviate the most from the ground truth, indicating that the SDA based on the Down sampler is crucial for accurate RUL prediction. By reducing the discrepancy in specific

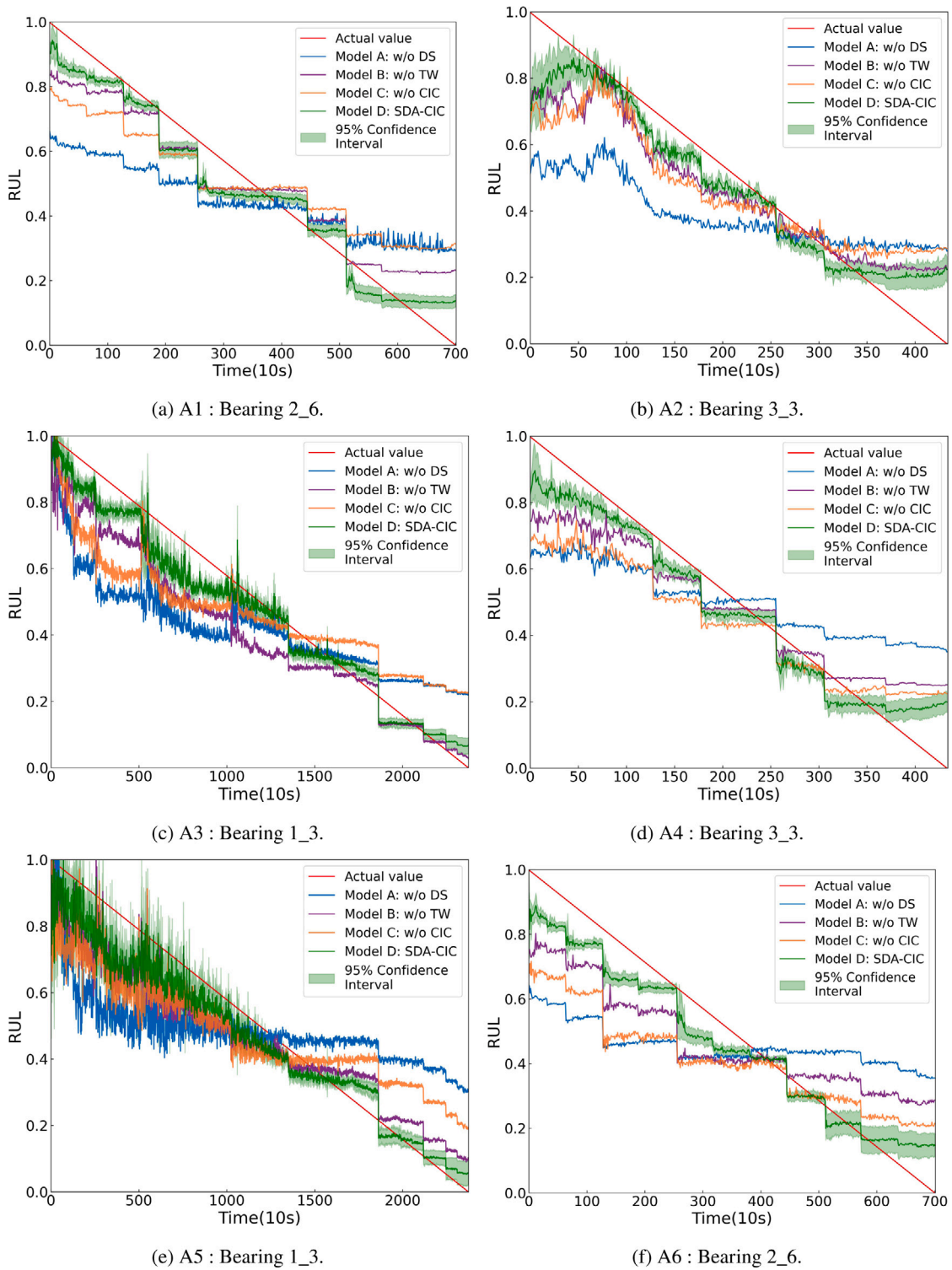


Fig. 9. Prediction results of Models A-D for the six transfer tasks in Table 3.

feature distributions within the source and target subdomains, the SDA network enhances RUL prediction performance in the target domain. Model B achieves the second-highest Score, just behind Model D, emphasizing the importance of the time-weighted mechanism. Without it, Model B treats all subdomains equally during domain alignment, failing to focus on key degradation-related features and thus limiting its ability to learn accurate RUL patterns. Model C performs better than Model A but worse than the others, suggesting that ‘slice-then-encode’

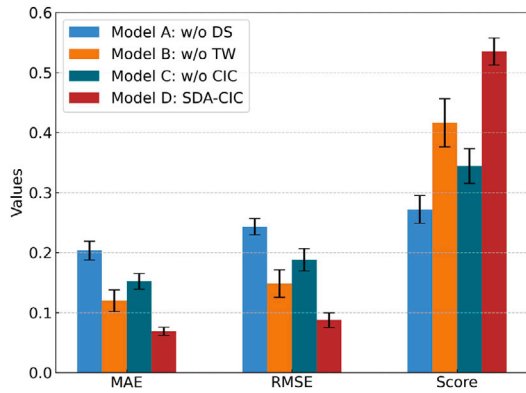
leads to the loss of contextual information, making it difficult for the Encoder to fully capture the degradation patterns in time series data. The introduction of the CIC network effectively solves this problem.

In task A4, although Model B has a lower MAE and a higher Score than Model C, its RMSE is unexpectedly higher. This is because RMSE is more sensitive to large errors, as it gives more weights to larger prediction errors, while MAE focuses on the average error level and is less affected by extreme errors. If a task has particularly large errors, its

Table 7

The average results compare the SDA-CIC with related models based on the PRONOSTIA dataset.

Method	Type	MAE	RMSE	Score
TCNN [12]	Metric-based DA	0.1962	0.2345	0.2874
MADA [30]	Adversarial-based DA	0.0796	0.0998	0.5014
TLMAN [13]	Classification network-based SDA	0.1877	0.2162	0.3033
DSAN-WM [14]	Classification network-based SDA	0.1487	0.1886	0.3253
DSARN [34]	Discretized continuous label-based SDA	0.1137	0.1411	0.3747
VLSTM-LWSAN [16]	Discretized continuous label-based SDA	0.0813	0.1065	0.4923
DSPC-A [32]	Discretized continuous label-based SDA	0.1050	0.1353	0.4462
HAMDN [38]	Metric-based SDA	0.1087	0.1369	0.4417
GSAN [39]	Metric-based SDA	0.0937	0.1154	0.4739
Proposed SDA-CIC	Metric-based SDA	0.0688	0.0874	0.5350

**Fig. 10.** The average RUL prediction metrics of the four models in Table 6.

RMSE will be high. In addition, these prediction curves all show different degrees of jumps. This is because bearing degradation typically goes through several stages rather than following a smooth and continuous trend. In real-world industrial environments, external shocks, load changes, or environmental disturbances often cause sudden accelerations in the degradation process. These abrupt degradations result in the transition of health status not being gradual but jumping-like. The Encoder successfully captures these abrupt degradation patterns during training, which are reflected as obvious jumps in the predicted RUL curves.

4.6. Comparison with related RUL prediction models

To demonstrate the superiority of the proposed SDA-CIC, it is compared with four types of related models: ① DA models, including the metric-based TCNN [12] and the adversarial-based MADA [30]; ② SDA models based on classification networks, such as TLMAN [13] and DSAN-WM [14]; ③ SDA models based on discretized continuous labels, including DSARN [34], VLSTM-LWSAN [16], and a modified version of DSPC-A [32]; ④ SDA models based on metrics, such as HAMDN [38], and GSAN [39]. All these models are evaluated using six transfer scenarios in Table 3. The average experimental results, comparing the proposed SDA-CIC with related RUL prediction models in terms of RMSE, MAE, and Score, are listed in Table 7. Furthermore, to intuitively compare the feature alignment and subdomain clustering capabilities of SDA models, T-SNE is employed to visualize the feature distributions of the source and target domain data in the last training epoch, as shown in Fig. 11.

The results in Table 7 reveal that the average prediction errors (MAE and RMSE) of SDA models are lower than those of DA models. This suggests that bearing degradation is a multi-stage process, where different stages have different feature distributions. DA models perform global domain alignment, but this is not sufficient for accurate prediction. SDA models improve prediction accuracy by aligning subdomains and

capturing local degradation features. However, the average prediction errors of the DA models are lower than those of classification network-based SDA models. This is because classification-based SDA follows a two-stage process, leading to error accumulation. In the first stage, the source domain data is divided into several subdomains to train the classifier, which may introduce segmentation errors. In the second stage, the predictor generates pseudolabels, which may result in misclassified errors. These two errors ultimately reduce the prediction accuracy.

In SDA models, discretized continuous label-based models show lower average errors than classification network-based models. This is because classification network-based models rely on statistical or clustering algorithms to predefine subdomains, introducing segmentation errors due to unclear degradation boundaries, as shown in Fig. 11(a) and (b). In contrast, discretized continuous label-based models divide the source domain data using evenly discretized labels, ensuring a stable and consistent subdomain division, as shown in Fig. 11(c), (d), and (e). However, the average errors of discretized continuous label-based models are higher than those of metric-based SDA models. This is because the source data is evenly divided into several discrete intervals; however, according to the visualization results shown in Fig. 11, the number of features contained in different subdomains is often uneven. Forced discretization may result in similar data points being assigned to different subdomains or dissimilar points being grouped together, increasing prediction errors. Additionally, the floor function used in the target domain prediction may introduce misclassified pseudolabel errors. To quantify the impact of misclassified pseudolabels on SDA, we introduce DSPC-A, a fault diagnosis model equipped with a pseudolabel correction mechanism. Specifically, the discretized continuous label strategy is incorporated into DSPC-A to divide the data into three subdomains. Its performance is then compared with that of HAMDN, which also uses three subdomains. As shown in Fig. 11(e) and (f), DSPC-A exhibits clearer subdomain boundaries and better prediction accuracy than HAMDN, underscoring the benefits of correcting pseudolabel errors in SDA.

Among metric-based SDA models, HAMDN and GSAN have higher prediction errors than SDA-CIC. This is because HAMDN and GSAN follow a two-stage structure. Although they reduce errors caused by manual subdomain division through adaptive subdomain matching, inaccurate similarity measurements in the first stage are directly propagated to the target domain prediction and cannot be optimized through subsequent training, leading to error accumulation. In contrast, the proposed SDA-CIC is an end-to-end model. Even if misalignment occurs during the subdomain matching process, linear layers in the Predictor still have the opportunity to correct the misalignment errors, thereby mitigating the impact of error propagation and ultimately reducing the prediction error. Additionally, unlike other SDA models, SDA-CIC achieves truly adaptive subdomain division by dynamically downsampling and aligning one subdomain at each epoch rather than predefining subdomains, as shown in Fig. 11(h).

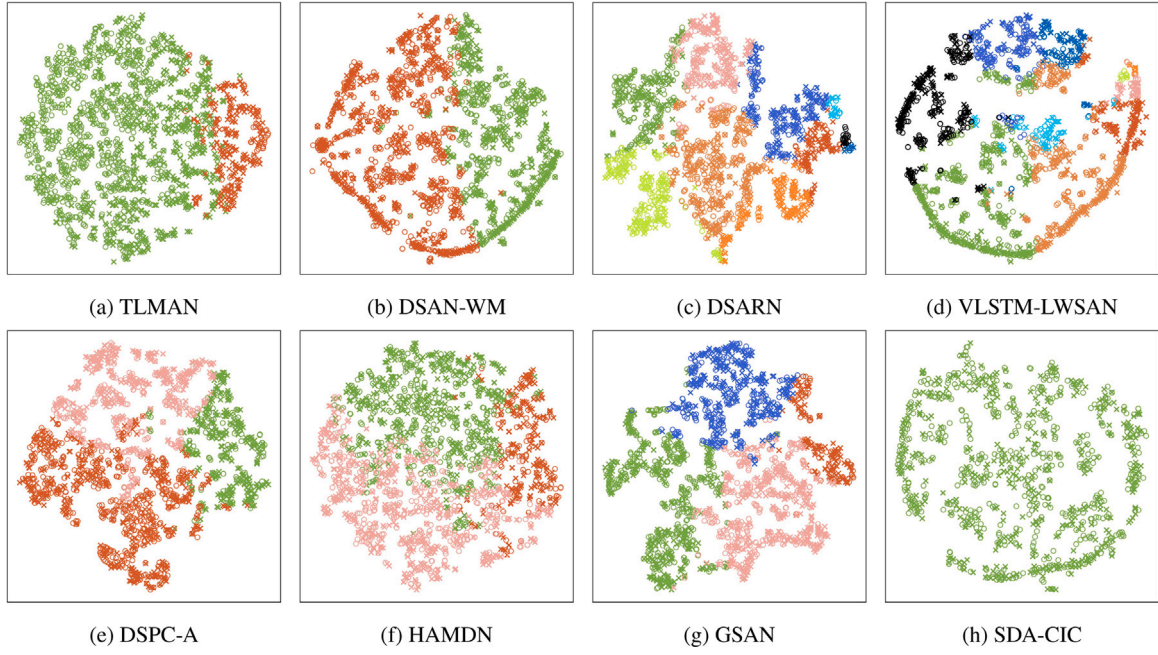


Fig. 11. T-SNE visualization of feature distributions for all SDA models listed in Table 7 in the last training epoch. 'O' and 'X' represent the source and target domains. Distinct colors indicate different subdomains, where (a) and (b) show 2 subdomains, (c) and (d) show 10 subdomains, (e) and (f) show 3 subdomains, (g) shows 4 subdomains, and (h) shows the alignment scenario of 1 subdomain that was downsampled in the final epoch.

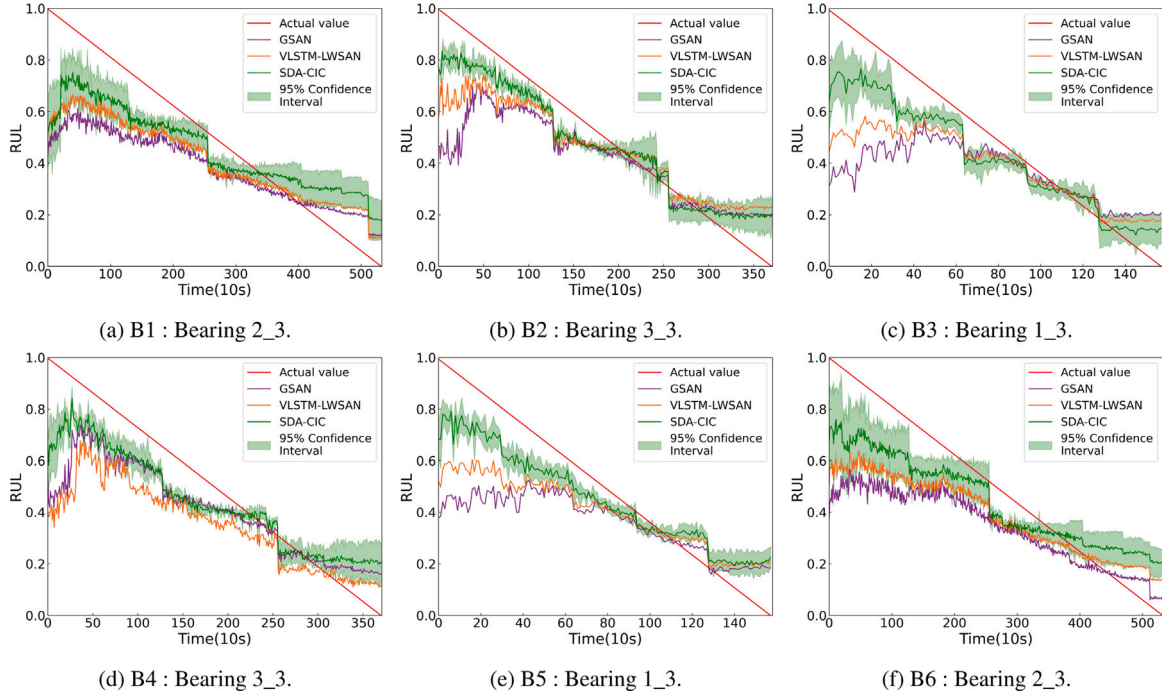


Fig. 12. Prediction results of SDA-CIC and two SOTA models across six transfer tasks in Table 3.

4.7. Validation of model generalization performance

To validate the generalization performance of the proposed SDA-CIC, the XJTU-SY dataset is also used for prediction. For validation, six transfer tasks are defined in Table 3, specifying the roles (training vs testing) of the various bearings in each task.

The performance of SDA-CIC is compared with four types of related models listed in Table 7. The prediction metrics are presented in Table 8, while their average results are shown in Table 9. To intuitively visualize the superiority of the proposed SDA-CIC, Fig. 12 illustrates the

prediction curves of SDA-CIC and two representative SOTA SDA-based models, namely VLSTM-LWSAN and GSAN.

According to Fig. 12, the prediction curves of SDA-CIC are closer to the ground truth than those of the two SOTA models, demonstrating its superior RUL prediction capability and strong generalization performance across datasets. As shown in Table 9, discretized continuous label-based SDA models outperform classification network-based SDA models, highlighting the advantage of end-to-end learning in reducing prediction errors. In addition, metric-based SDA models exhibit

Table 8

The RUL prediction metrics of four types of related models for the six transfer tasks in Table 3.

Models	Metrics	B1	B2	B3	B4	B5	B6
TCNN	MAE	0.2332 ± 0.0236	0.1709 ± 0.0275	0.2013 ± 0.0242	0.1801 ± 0.0314	0.1896 ± 0.0350	0.1947 ± 0.0283
	RMSE	0.2881 ± 0.0315	0.2158 ± 0.0292	0.2499 ± 0.0317	0.2223 ± 0.0311	0.2352 ± 0.0366	0.2561 ± 0.0344
	Score	0.2494 ± 0.0393	0.3341 ± 0.0485	0.2719 ± 0.0364	0.3362 ± 0.0436	0.3286 ± 0.0424	0.2810 ± 0.0435
MADA	MAE	0.1471 ± 0.0142	0.1064 ± 0.0121	0.1286 ± 0.0267	0.1423 ± 0.0201	0.1237 ± 0.0297	0.1329 ± 0.0187
	RMSE	0.1709 ± 0.0263	0.1235 ± 0.0272	0.1574 ± 0.0381	0.1721 ± 0.0316	0.1582 ± 0.0471	0.1594 ± 0.0520
	Score	0.3782 ± 0.0324	0.4264 ± 0.0326	0.3921 ± 0.0422	0.3819 ± 0.0402	0.3923 ± 0.0405	0.3917 ± 0.0562
TLMAN	MAE	0.2383 ± 0.0331	0.1481 ± 0.0168	0.1735 ± 0.0232	0.1487 ± 0.0157	0.1987 ± 0.0263	0.2137 ± 0.0293
	RMSE	0.2715 ± 0.0504	0.1764 ± 0.0206	0.2041 ± 0.0314	0.1747 ± 0.0287	0.2401 ± 0.0368	0.2547 ± 0.0388
	Score	0.2347 ± 0.0438	0.3749 ± 0.0651	0.3393 ± 0.0425	0.3796 ± 0.0431	0.2719 ± 0.0511	0.2527 ± 0.0426
DSAN-WM	MAE	0.1557 ± 0.0247	0.1226 ± 0.0263	0.1640 ± 0.0579	0.1543 ± 0.0145	0.1569 ± 0.0411	0.1542 ± 0.0305
	RMSE	0.1965 ± 0.0378	0.1558 ± 0.0476	0.2230 ± 0.0440	0.2032 ± 0.0277	0.2040 ± 0.0382	0.1955 ± 0.0456
	Score	0.3607 ± 0.0337	0.3916 ± 0.0509	0.3543 ± 0.0630	0.3625 ± 0.0332	0.3854 ± 0.0619	0.3659 ± 0.0402
DSARN	MAE	0.1707 ± 0.0113	0.1469 ± 0.0187	0.2043 ± 0.0376	0.1737 ± 0.0485	0.1860 ± 0.0532	0.1753 ± 0.0195
	RMSE	0.2153 ± 0.0228	0.2029 ± 0.0474	0.2862 ± 0.0665	0.2223 ± 0.0661	0.2535 ± 0.0589	0.2258 ± 0.0838
	Score	0.3378 ± 0.0279	0.3789 ± 0.0264	0.2594 ± 0.0297	0.3396 ± 0.0829	0.3298 ± 0.0499	0.3336 ± 0.0308
VLSTM-LWSAN	MAE	0.1502 ± 0.0235	0.1125 ± 0.0220	0.1380 ± 0.0241	0.1448 ± 0.0250	0.1239 ± 0.0234	0.1483 ± 0.0229
	RMSE	0.1711 ± 0.0310	0.1299 ± 0.0285	0.1582 ± 0.0309	0.1723 ± 0.0332	0.1597 ± 0.0298	0.1688 ± 0.0307
	Score	0.3663 ± 0.0563	0.3987 ± 0.0621	0.3845 ± 0.0700	0.3801 ± 0.0392	0.3857 ± 0.0610	0.3774 ± 0.0362
DSPC-A	MAE	0.1694 ± 0.0147	0.1132 ± 0.0131	0.1456 ± 0.0168	0.1438 ± 0.0142	0.1684 ± 0.0172	0.1342 ± 0.0123
	RMSE	0.1911 ± 0.0294	0.1497 ± 0.0213	0.1645 ± 0.0206	0.1793 ± 0.0282	0.2078 ± 0.0379	0.1609 ± 0.0290
	Score	0.3547 ± 0.0389	0.4074 ± 0.0327	0.3763 ± 0.0251	0.3802 ± 0.0414	0.3529 ± 0.0502	0.3899 ± 0.0434
HAMDN	MAE	0.1473 ± 0.0221	0.1284 ± 0.0195	0.1359 ± 0.0272	0.1630 ± 0.0387	0.1299 ± 0.0204	0.1415 ± 0.0213
	RMSE	0.1804 ± 0.0385	0.1555 ± 0.0201	0.1717 ± 0.0366	0.1990 ± 0.0553	0.1689 ± 0.0447	0.1860 ± 0.0261
	Score	0.3715 ± 0.0412	0.3967 ± 0.0337	0.3801 ± 0.0588	0.3665 ± 0.0526	0.3899 ± 0.0551	0.3702 ± 0.0303
GSAN	MAE	0.1562 ± 0.0289	0.1099 ± 0.0249	0.1286 ± 0.0262	0.1549 ± 0.0291	0.1337 ± 0.0274	0.1511 ± 0.0315
	RMSE	0.1977 ± 0.0352	0.1346 ± 0.0273	0.1587 ± 0.0309	0.1918 ± 0.0337	0.1614 ± 0.0288	0.1850 ± 0.0336
	Score	0.3594 ± 0.0814	0.4092 ± 0.0485	0.3905 ± 0.0625	0.3652 ± 0.0457	0.3905 ± 0.0473	0.3675 ± 0.0698
SDA-CIC	MAE	0.1349 ± 0.0155	0.0822 ± 0.0109	0.1036 ± 0.0134	0.1329 ± 0.0201	0.1070 ± 0.0147	0.1268 ± 0.0221
	RMSE	0.1680 ± 0.0243	0.0979 ± 0.0340	0.1282 ± 0.0217	0.1660 ± 0.0239	0.1279 ± 0.0257	0.1540 ± 0.0322
	Score	0.3936 ± 0.0371	0.4922 ± 0.0298	0.4516 ± 0.0323	0.3957 ± 0.0439	0.4381 ± 0.0346	0.4026 ± 0.0467

Table 9

Validation results in comparison with related methods averaged over six transfer tasks in Table 8.

Method	Type	MAE	RMSE	Score
TCNN [12]	Metric-based DA	0.1951	0.2446	0.3002
MADA [30]	Adversarial-based DA	0.1296	0.1569	0.3942
TLMAN [13]	Classification network-based SDA	0.1868	0.2203	0.3088
DSAN-WM [14]	Classification network-based SDA	0.1513	0.1963	0.3701
DSARN [34]	Discretized continuous label-based SDA	0.1761	0.2343	0.3299
VLSTM-LWSAN [16]	Discretized continuous label-based SDA	0.1363	0.1600	0.3821
DSPC-A [32]	Discretized continuous label-based SDA	0.1458	0.1755	0.3769
HAMDN [38]	Metric-based SDA	0.1410	0.1769	0.3792
GSAN [39]	Metric-based SDA	0.1391	0.1715	0.3807
Proposed SDA-CIC	Metric-based SDA	0.1146	0.1403	0.4290

lower average errors than discretized continuous label-based models, confirming that misclassified pseudolabels negatively impact prediction accuracy. In addition, these prediction curves show different degrees of fluctuations. Such fluctuations are consistent with the actual physical degradation behavior of bearings, and they also demonstrate the SDA-CIC's ability to learn and represent realistic degradation patterns.

5. Discussion

In this section, we discuss several important components of our method, each of which plays a crucial role in achieving accurate RUL prediction.

The necessity of pre-training the Encoder. Pre-training the Encoder offers two main advantages. Firstly, using the triplet loss enables the Encoder to learn discriminative feature representations by minimizing the distance between positive pairs and maximizing the distance between negative pairs, even in the absence of labeled data. During the main training stage, if the Encoder has been pretrained, it can provide high-quality and informative representations, which allow the SDA-CIC

to perform accurate subdomain alignment during joint optimization. Secondly, the pre-training stage allows the use of a larger dataset to further improve model performance. Since the main objective of pre-training is to learn the ability to compensate for contextual information loss rather than optimizing other downstream tasks, the Encoder can benefit from exposure to a broader and more diverse dataset. This leads to more generalized and robust feature representations that can better support subsequent learning objectives.

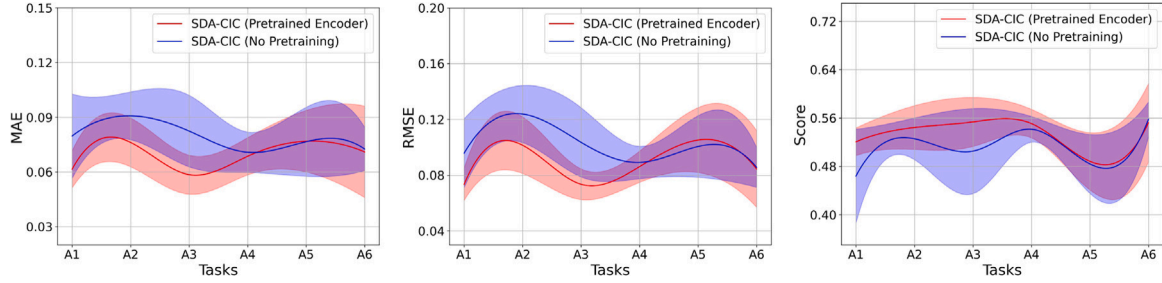
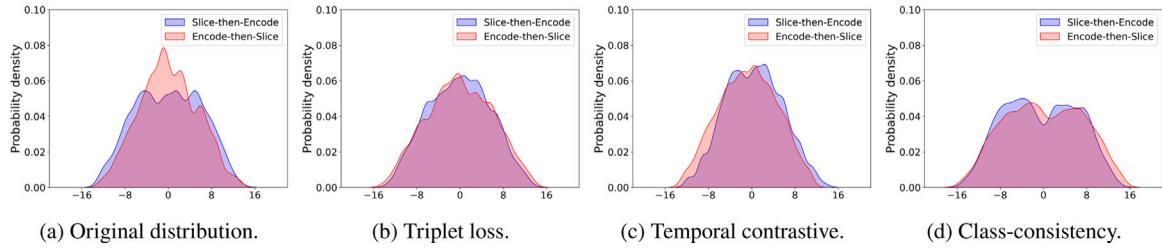
Table 10 presents the prediction metrics of the SDA-CIC with and without pre-training across six transfer tasks, as defined in Table 3. In order to visualize the improvements in prediction performance brought by the pretrained Encoder, Fig. 13 shows the error band curves for MAE, RMSE and Score.

Choice of self-supervision strategies. The 'slice-then-encode' process shown in Fig. 5 limits the amount of contextual information available in local segments. To address this issue, a self-supervised learning network should be introduced to help the Encoder generate informative representations from data segments. The core idea is to align the representations of local segments with their counterparts extracted

Table 10

The RUL prediction metrics of the SDA-CIC with and without pre-training for the six transfer tasks in Table 3.

Models	Metrics	A1	A2	A3	A4	A5	A6
SDA-CIC (Pretrained)	MAE	0.0615 ± 0.0102	0.0762 ± 0.0134	0.0585 ± 0.0107	0.0686 ± 0.0102	0.0769 ± 0.0169	0.0710 ± 0.0251
	RMSE	0.0734 ± 0.0116	0.1015 ± 0.0203	0.0735 ± 0.0110	0.0861 ± 0.0112	0.1052 ± 0.0233	0.0844 ± 0.0275
	Score	0.5207 ± 0.0229	0.5443 ± 0.0362	0.5536 ± 0.0404	0.5508 ± 0.0237	0.4883 ± 0.0474	0.5524 ± 0.0648
SDA-CIC (Not Pretrained)	MAE	0.0797 ± 0.0231	0.0908 ± 0.0131	0.0824 ± 0.0196	0.0708 ± 0.0111	0.0766 ± 0.0189	0.0726 ± 0.0118
	RMSE	0.0957 ± 0.0247	0.1239 ± 0.0205	0.1036 ± 0.0248	0.0891 ± 0.0116	0.1006 ± 0.0221	0.0856 ± 0.0143
	Score	0.4640 ± 0.0775	0.5256 ± 0.0346	0.5053 ± 0.0704	0.5414 ± 0.0212	0.4839 ± 0.0488	0.5582 ± 0.0285

**Fig. 13.** Prediction metrics of the SDA-CIC with and without pretrained Encoder.**Fig. 14.** Probability density curves of r_{sub}^S generated by the 'slice-then-encode' and p^S obtained by the 'encode-then-slice'. (a) shows the curves before compensation; (b)–(d) show the curves after compensation.

from the full-sequence encoding. In this context, we compare three self-supervised strategies that aim to reduce the discrepancy between local and global representations: ① the proposed triplet loss (Eq. (7)); ② the temporal contrastive loss proposed by Yue et al. [41]; and ③ the class-consistency constraint proposed by Li et al. [25].

The temporal contrastive loss $\mathcal{L}_{\text{temporal}}$ involves first constructing an augmented view and then generating timestamp-level positive-negative pairs within this view to optimize the objective function. We treat r_{sub}^S and p^S as two augmented views extracted from the same position index. The goal of this strategy is to maximize the similarity between local and global representations at the same timestamp, while distinguishing them from representations at different timestamps. Specifically, for each timestamp t , we consider $(r_{\text{sub},t}^S, p_t^S)$ as a positive pair, and $(r_{\text{sub},t}^S, p_{t'}^S)$ ($t' \neq t$) as negative pairs. The temporal contrast loss $\mathcal{L}_{\text{temporal}}$ is defined as:

$$\mathcal{L}_{\text{temporal}} = -\log \frac{\exp(r_{\text{sub},t}^S \cdot p_t^S)}{\sum_{t' \in \Omega} \left\{ \exp(r_{\text{sub},t}^S \cdot p_{t'}^S) + \mathbb{I}_{t \neq t'} \exp(r_{\text{sub},t}^S \cdot r_{\text{sub},t'}^S) \right\}}, \quad (14)$$

where Ω is the set of time steps within the temporal range of r_{sub}^S and p^S , and $\mathbb{I}$ is the indicator function.

Inspired by the class-consistency constraint $\mathcal{L}_{\text{consist}}$, we regard r_{sub}^S generated by the 'slice-then-encode' path and p^S obtained by the 'encode-then-slice' path as 'neighbors' because they share the same position index. The goal of this strategy is to compensate for the contextual information loss by maximizing the similarity between these neighboring representations. $\mathcal{L}_{\text{consist}}$ measures the similarity using

cosine similarity, and is defined as:

$$\mathcal{L}_{\text{consist}} = 1 - \frac{\mathbf{r}_{\text{sub}}^S \cdot \mathbf{p}^S}{\|\mathbf{r}_{\text{sub}}^S\| \|\mathbf{p}^S\|}, \quad (15)$$

To visualize the effectiveness of contextual information compensation, we train the Encoder using the above three self-supervised strategies, respectively. Fig. 14 shows the probability density curves of r_{sub}^S generated by the 'slice-then-encode' and p^S obtained by the 'encode-then-slice'. All three strategies demonstrate a clear alignment between r_{sub}^S and p^S , indicating their ability to effectively recover contextual information. We ultimately chose the triplet loss because it is conceptually compatible with the overall model architecture.

Choice of model structure in encoder. RUL prediction requires the Encoder to effectively capture both local degradation patterns and long-term temporal dependencies from long sequence data. While LSTM and Transformer have demonstrated success in time-series modeling, the proposed dilated CNN may be more suitable for industrial scenarios. We compare these three model structures in the following.

LSTM is designed to process sequence data with long-term dependencies. However, it processes data at each time point step by step and cannot process multiple time points in parallel, so the calculation speed is slow and the efficiency is low. In addition, due to the gradient vanishing problem, LSTM may have difficulty in capturing complex degradation patterns in long sequences. In contrast, dilated CNN can capture both short-term and long-term dependencies efficiently through its exponentially expanding receptive field, while allowing parallel processing and faster computation [46].

Transformer leverages the self-attention mechanism to capture global dependencies by computing the relationships between each time

step and all others in the sequence. However, this results in high computational resource requirements. Specifically, given a sequence of length L , the computational complexity of self-attention is $\mathcal{O}(L^2)$. In contrast, dilated CNN performs local operations within a limited receptive field, with a linear complexity of $\mathcal{O}(L)$, making it significantly faster and more efficient [47].

Real-world application potential. The proposed model exhibits several advantages that highlight its potential for deployment in real industrial scenarios. First, it possesses cross-domain transferability, enabling robust RUL prediction under varying working conditions without requiring labeled testing data. Second, it performs fine-grained subdomain alignment, effectively addressing feature distribution discrepancies commonly encountered in complex industrial systems. Finally, the model adopts lightweight and modular architecture, making it easy to integrate into existing predictive maintenance frameworks. In practice, we can use historical maintenance records to perform weakly supervised training based on the ‘degradation time before maintenance’ as the approximate RUL of the equipment. For example, if the equipment gradually degrades over the 6 months before maintenance, the RUL during this period can be considered to decrease from 1 to 0. Once labels are constructed, the model can be trained and used to infer RUL.

6. Conclusion

This paper proposes a SDA-CIC model, including a backbone RUL prediction network, a SDA network, and a CIC network. The results of our ablation, comparison, and generalization experiments based on two datasets demonstrate several key findings: ① SDA-CIC effectively captures fine-grained degradation information within subdomains. ② SDA-CIC adaptively divides subdomains for precise SDA. ③ SDA-CIC consistently outperforms the existing advanced models.

For future work, a promising direction for extending the capabilities of the SDA-CIC model involves incorporating multi-source domain generalization. By simultaneously learning from multiple source domains, the model could better capture the variability across different conditions, enhancing its adaptability to new and diverse conditions. Multi-source learning frameworks offer the potential to make the model more robust, allowing it to generalize more effectively to unseen operating scenarios.

CRedit authorship contribution statement

Zhiqing Xu: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization. **Yee Wei Law:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation. **Md Mizanur Rahman:** Writing – review & editing, Supervision, Funding acquisition. **Raufdeen Rameezdeen:** Supervision, Funding acquisition. **Christopher W.K. Chow:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

SA Water funded Zhiqing Xu’s scholarship under Project Number SW100598.

Data availability

Data will be made available on request.

References

- [1] Z. Chen, H. Zhu, J. Wu, L. Fan, Health indicator construction for degradation assessment by embedded LSTM–CNN autoencoder and growing self-organized map, *Knowl.-Based Syst.* 252 (2022) 109399, <http://dx.doi.org/10.1016/j.knsys.2022.109399>.
- [2] O.V. Thorsen, M. Dalva, A survey of faults on induction motors in offshore oil industry, petrochemical industry, gas terminals, and oil refineries, *IEEE Trans. Ind. Appl.* 31 (5) (1995) 1186–1196, <http://dx.doi.org/10.1109/28.464536>.
- [3] S. Suh, P. Lukowicz, Y.O. Lee, Generalized multiscale feature extraction for remaining useful life prediction of bearings with generative adversarial networks, *Knowl.-Based Syst.* 237 (2022) 107866, <http://dx.doi.org/10.1016/j.knsys.2021.107866>.
- [4] Y. Chang, J. Chen, Y. Liu, E. Xu, S. He, Temporal convolution-based sorting feature repeat-explore network combining with multi-band information for remaining useful life estimation of equipment, *Knowl.-Based Syst.* 249 (2022) 108958, <http://dx.doi.org/10.1016/j.knsys.2022.108958>.
- [5] F. Gan, H. Shao, B. Xia, An adaptive model with dual-dimensional attention for remaining useful life prediction of aero-engine, *Knowl.-Based Syst.* 293 (2024) 111738, <http://dx.doi.org/10.1016/j.knsys.2024.111738>.
- [6] M. Gupta, R. Wadhvani, A. Rasool, A real-time adaptive model for bearing fault classification and remaining useful life estimation using deep neural network, *Knowl.-Based Syst.* 259 (2023) 110070, <http://dx.doi.org/10.1016/j.knsys.2022.110070>.
- [7] B. Wang, Y. Lei, N. Li, T. Yan, Deep separable convolutional network for remaining useful life prediction of machinery, *Mech. Syst. Signal Process.* 134 (2019) 106330, <http://dx.doi.org/10.1016/j.ymssp.2019.106330>.
- [8] S. Wan, X. Li, Y. Zhang, S. Liu, J. Hong, D. Wang, Bearing remaining useful life prediction with convolutional long short-term memory fusion networks, *Reliab. Eng. Syst. Saf.* 224 (2022) 108528, <http://dx.doi.org/10.1016/j.res.2022.108528>.
- [9] Z. Zhang, H. Gao, W. Sun, W. Song, Q. Li, Multivariate time series generation based on dual-channel transformer conditional GAN for industrial remaining useful life prediction, *Knowl.-Based Syst.* 308 (2025) 112749, <http://dx.doi.org/10.1016/j.knsys.2024.112749>.
- [10] V. Warke, S. Kumar, A. Bongale, K. Kotecha, Robust tool wear prediction using multi-sensor fusion and time-domain features for the milling process using instance-based domain adaptation, *Knowl.-Based Syst.* 288 (2024) 111454, <http://dx.doi.org/10.1016/j.knsys.2024.111454>.
- [11] Y. Ding, M. Jia, Y. Cao, P. Ding, X. Zhao, C.-G. Lee, Domain generalization via adversarial out-domain augmentation for remaining useful life prediction of bearings under unseen conditions, *Knowl.-Based Syst.* 261 (2023) 110199, <http://dx.doi.org/10.1016/j.knsys.2022.110199>.
- [12] H. Cheng, X. Kong, G. Chen, Q. Wang, R. Wang, Transferable convolutional neural network based remaining useful life prediction of bearing under multiple failure behaviors, *Measurement* 168 (2021) 108286, <http://dx.doi.org/10.1016/j.measurement.2020.108286>.
- [13] S. Dong, J. Xiao, X. Hu, N. Fang, L. Liu, J. Yao, Deep transfer learning based on Bi-LSTM and attention for remaining useful life prediction of rolling bearing, *Reliab. Eng. Syst. Saf.* 230 (2023) 108914, <http://dx.doi.org/10.1016/j.res.2022.108914>.
- [14] C. Wu, J. He, W. Shen, W. Xu, S. Liu, et al., Remaining useful life prediction across operating conditions based on deep subdomain adaptation network considering the weighted multi-source domain, *Knowl.-Based Syst.* 301 (2024) 112291, <http://dx.doi.org/10.1016/j.knsys.2024.112291>.
- [15] Y. Ding, M. Jia, Q. Miao, P. Huang, Remaining useful life estimation using deep metric transfer learning for kernel regression, *Reliab. Eng. Syst. Saf.* 212 (2021) 107583, <http://dx.doi.org/10.1016/j.res.2021.107583>.
- [16] J. Zhang, X. Li, J. Tian, Y. Jiang, H. Luo, S. Yin, A variational local weighted deep sub-domain adaptation network for remaining useful life prediction facing cross-domain condition, *Reliab. Eng. Syst. Saf.* 231 (2023) 108986, <http://dx.doi.org/10.1016/j.res.2022.108986>.
- [17] H. Shi, S. Cao, H. Zuo, J. Ma, C. Lin, Deep subdomain adversarial network with self-supervised learning for aero-engine high speed bearing fault diagnosis with unknown working conditions, *Measurement* 241 (2025) 115668, <http://dx.doi.org/10.1016/j.measurement.2024.115668>.
- [18] M. Zhang, C. He, C. Huang, J. Yang, A weighted time embedding transformer network for remaining useful life prediction of rolling bearing, *Reliab. Eng. Syst. Saf.* 251 (2024) 110399, <http://dx.doi.org/10.1016/j.res.2024.110399>.
- [19] Q. Zheng, P. Teng, K. Zhang, G. Ding, X. Lai, Z. Li, Z. Yuan, A generalized network with domain invariance and specificity representation for bearing remaining useful life prediction under unknown conditions, *Knowl.-Based Syst.* 310 (2025) 112915, <http://dx.doi.org/10.1016/j.knsys.2024.112915>.
- [20] K. Li, M. Chen, Y. Lin, Z. Li, X. Jia, B. Li, A novel adversarial domain adaptation transfer learning method for tool wear state prediction, *Knowl.-Based Syst.* 254 (2022) 109537, <http://dx.doi.org/10.1016/j.knsys.2022.109537>.
- [21] J. Chen, R. Huang, Z. Chen, W. Mao, W. Li, Transfer learning algorithms for bearing remaining useful life prediction: A comprehensive review from an industrial application perspective, *Mech. Syst. Signal Process.* 193 (2023) 110239, <http://dx.doi.org/10.1016/j.ymssp.2023.110239>.

- [22] Y. Ding, M. Jia, Q. Miao, P. Huang, Remaining useful life estimation using deep metric transfer learning for kernel regression, *Reliab. Eng. Syst. Saf.* 212 (2021) 107583, <http://dx.doi.org/10.1016/j.res.2021.107583>.
- [23] J. Zhuang, M. Jia, Y. Ding, P. Ding, Temporal convolution-based transferable cross-domain adaptation approach for remaining useful life estimation under variable failure behaviors, *Reliab. Eng. Syst. Saf.* 216 (2021) 107946, <http://dx.doi.org/10.1016/j.res.2021.107946>.
- [24] Y. Cao, M. Jia, P. Ding, Y. Ding, Transfer learning for remaining useful life prediction of multi-conditions bearings based on bidirectional-GRU network, *Measurement* 178 (2021) 109287, <http://dx.doi.org/10.1016/j.measurement.2021.109287>.
- [25] H. Li, Z. Liu, J. Lin, J. Jiao, T. Zhang, W. Li, Anti-forgetting source-free domain adaptation method for machine fault diagnosis, *Knowl.-Based Syst.* 305 (2024) 112672, <http://dx.doi.org/10.1016/j.knsys.2024.112672>.
- [26] W. Mao, J. Chen, J. Liu, X. Liang, Self-supervised deep domain-adversarial regression adaptation for online remaining useful life prediction of rolling bearing under unknown working condition, *IEEE Trans. Ind. Inform.* 19 (2) (2023) 1227–1237, <http://dx.doi.org/10.1109/TII.2022.3172704>.
- [27] P.R. de Oliveira da Costa, A. Akçay, Y. Zhang, U. Kaymak, Remaining useful lifetime prediction via deep domain adaptation, *Reliab. Eng. Syst. Saf.* 195 (2020) 106682, <http://dx.doi.org/10.1016/j.res.2019.106682>.
- [28] X. Dong, C. Zhang, H. Liu, D. Wang, T. Wang, A multi-constrained domain adaptation network for remaining useful life prediction of bearings, *Mech. Syst. Signal Process.* 206 (2024) 110900, <http://dx.doi.org/10.1016/j.ymssp.2023.110900>.
- [29] M. Furqon, M. Pratama, L. Liu, H. Habibullah, K. Dogancay, Mixup domain adaptations for dynamic remaining useful life predictions, *Knowl.-Based Syst.* 295 (2024) 111783, <http://dx.doi.org/10.1016/j.knsys.2024.111783>.
- [30] J. Zhuang, M. Jia, X. Zhao, An adversarial transfer network with supervised metric for remaining useful life prediction of rolling bearing under multiple working conditions, *Reliab. Eng. Syst. Saf.* 225 (2022) 108599, <http://dx.doi.org/10.1016/j.res.2022.108599>.
- [31] J. Liu, L. Wan, F. Xie, Y. Sun, X. Wang, D. Li, S. Wu, Cross-machine deep subdomain adaptation network for wind turbines fault diagnosis, *Mech. Syst. Signal Process.* 210 (2024) 111151, <http://dx.doi.org/10.1016/j.ymssp.2024.111151>.
- [32] C. Li, H. Wang, T. Han, Dynamic subdomain pseudolabel correction and adaptation framework for multiscenario mechanical fault diagnosis, *IEEE Trans. Reliab.* (2024) <http://dx.doi.org/10.1109/TR.2024.3397913>.
- [33] X. Liu, Y. Lei, N. Li, X. Li, B. Yang, An optimal-subdomain generalization method for remaining useful life prediction of machinery under time-varying operation conditions, *IEEE Trans. Ind. Inform.* 20 (4) (2024) 6177–6186, <http://dx.doi.org/10.1109/TII.2023.3342885>.
- [34] Y. Ding, M. Jia, Y. Cao, Remaining useful life estimation under multiple operating conditions via deep subdomain adaptation, *IEEE Trans. Instrum. Meas.* 70 (2021) 1–11, <http://dx.doi.org/10.1109/TIM.2021.3076567>.
- [35] Y. Lyu, Z. Shen, N. Zhou, Z. Wen, C. Chen, A feature separation transfer network with contrastive metric for remaining useful life prediction under different working conditions, *Reliab. Eng. Syst. Saf.* 256 (2025) 110790, <http://dx.doi.org/10.1016/j.res.2024.110790>.
- [36] J. Yan, Z.-S. Ye, S. He, Z. He, An unsupervised subdomain adaptation of cross-domain remaining useful life prediction for sensor-equipped equipments, *Comput. Ind. Eng.* 203 (2025) 110967, <http://dx.doi.org/10.1016/j.cie.2025.110967>.
- [37] H.-W. Lu, C.-Y. Lee, Feature-enhanced multisource subdomain adaptation on robust remaining useful life prediction, *IEEE Robot. Autom. Lett.* (2024) <http://dx.doi.org/10.1109/LRA.2024.3400160>.
- [38] H.-B. Zhang, D.-J. Cheng, K.-L. Zhou, S.-W. Zhang, Deep transfer learning-based hierarchical adaptive remaining useful life prediction of bearings considering the correlation of multistage degradation, *Knowl.-Based Syst.* 266 (2023) 110391, <http://dx.doi.org/10.1016/j.knsys.2023.110391>.
- [39] J. Zhuang, Y. Chen, X. Zhao, M. Jia, K. Feng, A graph-embedded subdomain adaptation approach for remaining useful life prediction of industrial IoT systems, *IEEE Internet Things J.* 11 (13) (2024) 22903–22914, <http://dx.doi.org/10.1109/JIOT.2024.3361533>.
- [40] O. Yazdanbakhsh, S. Dick, Multivariate time series classification using dilated convolutional neural network, 2019, <http://dx.doi.org/10.48550/arXiv.1905.01697>, arXiv preprint arXiv:1905.01697.
- [41] Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, B. Xu, TS2Vec: Towards universal representation of time series, *Proc. the AAAI Conf. Artif. Intell.* 36 (8) (2022) 8980–8987, <http://dx.doi.org/10.1609/aaai.v36i8.20881>.
- [42] J.-Y. Franceschi, A. Dieuleveut, M. Jaggi, Unsupervised scalable representation learning for multivariate time series, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [43] F. Schroff, D. Kalenichenko, J. Philbin, Facenet: A unified embedding for face recognition and clustering, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 815–823.
- [44] P. Nectoux, R. Gouriveau, K. Medjaher, E. Ramasso, B. Chebel-Morello, N. Zerhouni, C. Varnier, PRONOSTIA: An experimental platform for bearings accelerated degradation tests, in: *IEEE International Conference on Prognostics and Health Management, PHM'12*, 2012.
- [45] B. Wang, Y. Lei, N. Li, N. Li, A hybrid prognostics approach for estimating remaining useful life of rolling element bearings, *IEEE Trans. Reliab.* 69 (1) (2020) 401–412, <http://dx.doi.org/10.1109/TR.2018.2882682>.
- [46] Y. Wang, Y. Han, H. Wang, X. Zhang, Contrast everything: A hierarchical contrastive framework for medical time-series, *Adv. Neural Inf. Process. Syst.* 36 (2023) 55694–55717.
- [47] J. Liu, S. Chen, Timesurl: Self-supervised contrastive learning for universal time series representation learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, pp. 13918–13926, <http://dx.doi.org/10.1609/aaai.v38i12.29299>.